Université de Jendouba
Institut Supérieur des Langues Appliquées et Informatique de Béja

Module:

Framework web Python



2MDS/ A.U. 2026-2027

Enseignant responsable: Nabil KHEMIRI, E-mail: nabil.khemiri@gmail.com

Chapitre 1 : Rappel sur les notions de base

Définition

Framework web

Un framework (ou « cadre de travail ») est un ensemble :

- **d'outils (générateurs de code, outils de débogage, gestionnaires de base de données, …)**,
- **de règles (rangement des fichiers, normes d'écriture,…)** et
- de **composants logiciels (module d'authentification, système de sécurité (filtres automatique),…)**

prêts à l'emploi qui sert de base pour créer des applications.

Objectifs : créer des applications d'une manière rapide, évolutive, facilement maintenable avec des composants réutilisables et sécurisée.

Framework web Django

Caractéristiques principales :

- **Architecture MVT :** Utilise le modèle MVT pour séparer la logique de données, la logique métier et l'affichage. Impose une organisation du code standardisée (structure claire) et facile à maintenir.

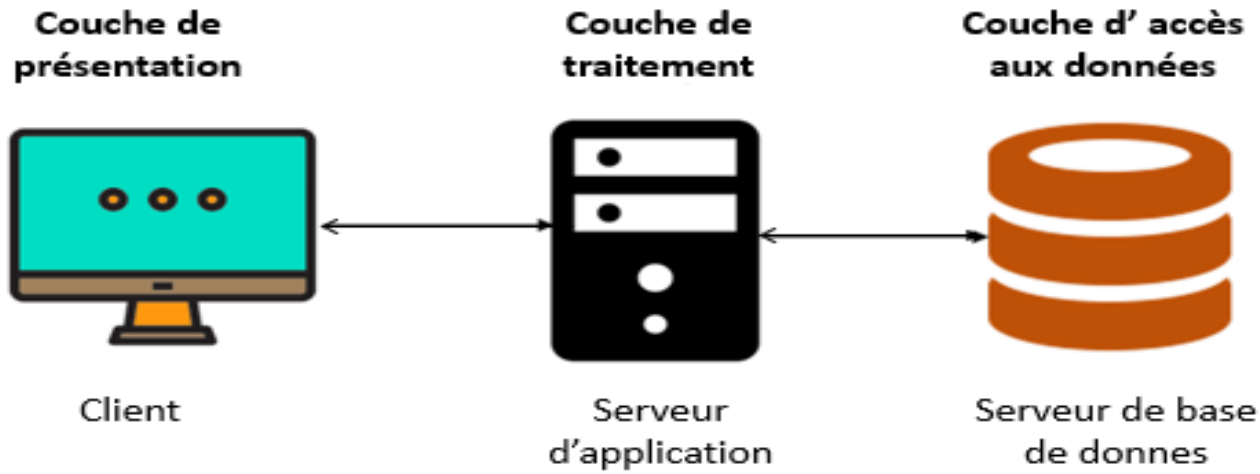
- **Template incluses :** Intègre nativement un panneau d'administration, un système d'authentification, un ORM (Object-Relational Mapping) pour gérer les bases de données et une protection contre les failles de sécurité courantes.

- **Efficacité :** Permet d'écrire moins de code grâce à la réutilisation et de se concentrer sur la création de l'application.

1.1 Architecture Physique VS architecture logique

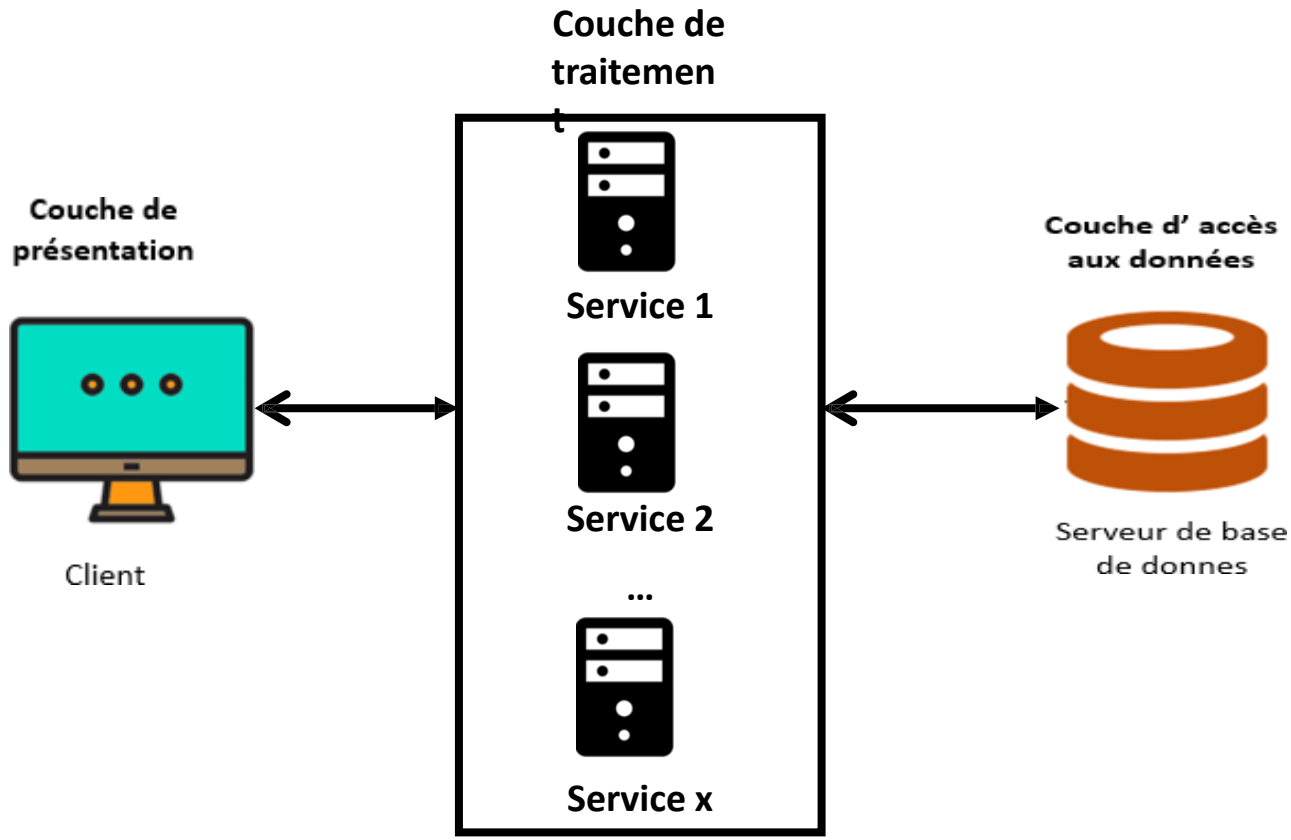
Architecture physique

- **L'architecture physique (ou technique)** décrit **l'ensemble des composants matériels** et **des infrastructures concrètes** qui supportent un système ou une application informatique.



Architecture 3 tiers

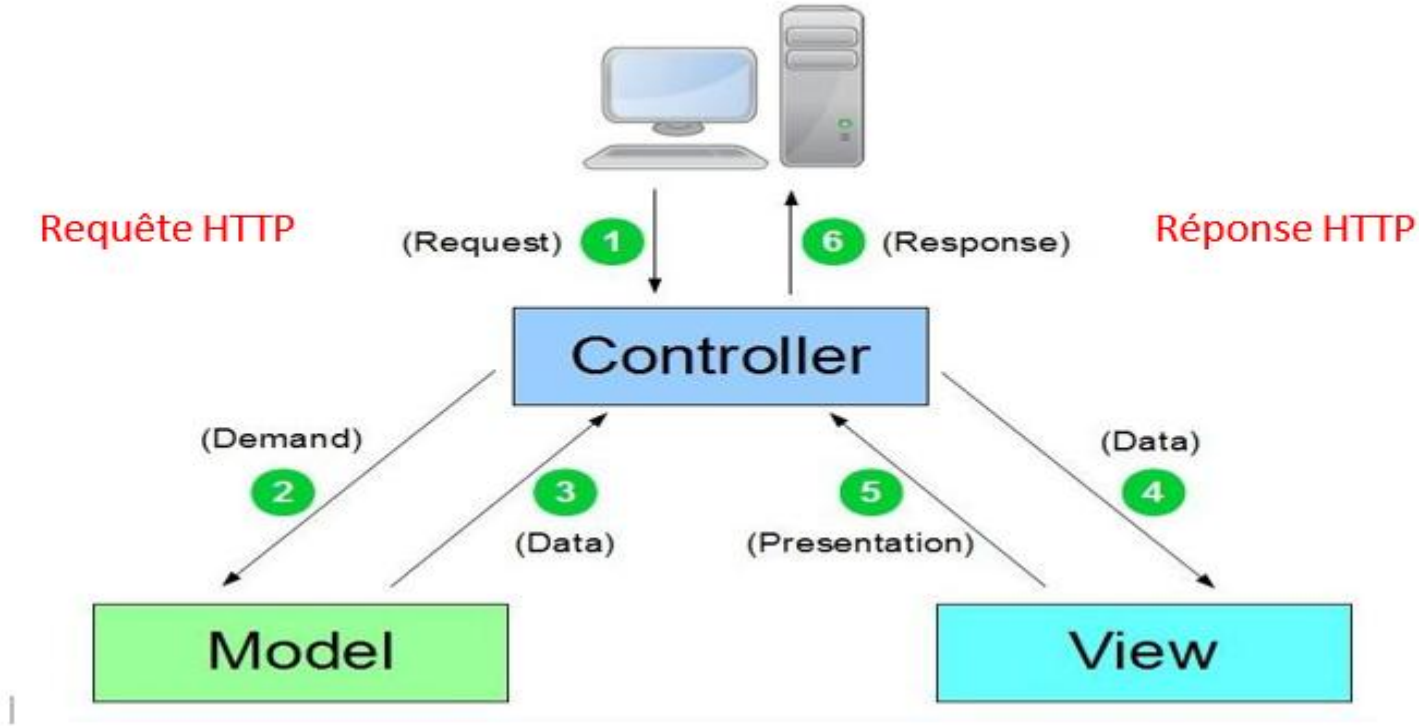
Architecture physique



Architecture n-tiers (multi-Tiers)

Architecture logique

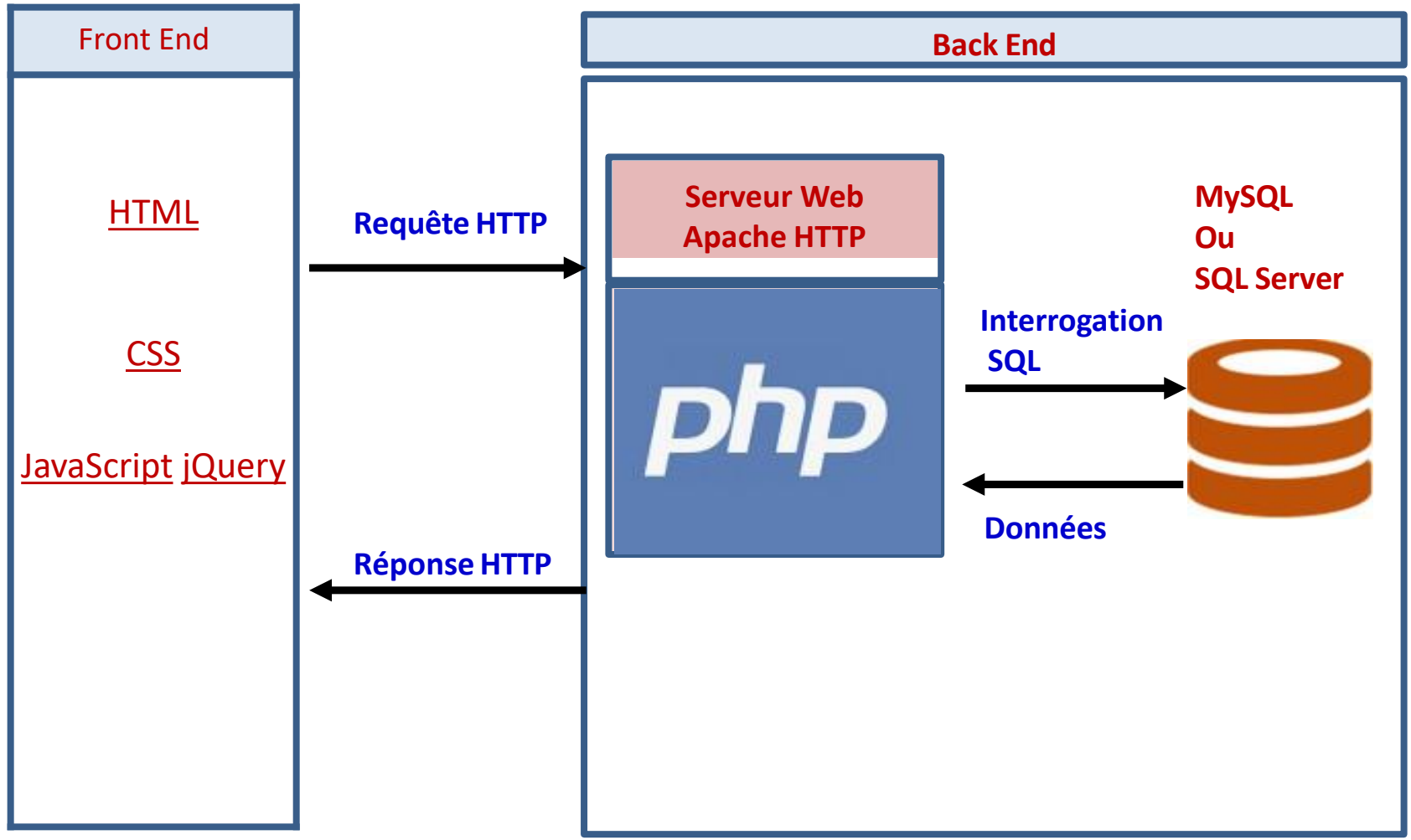
- **L'architecture logique** décrit **l'organisation fonctionnelle des composants** d'un système, de **ses services** et de **ses flux de données** sans se soucier du matériel ou des technologies physiques sous-jacentes.



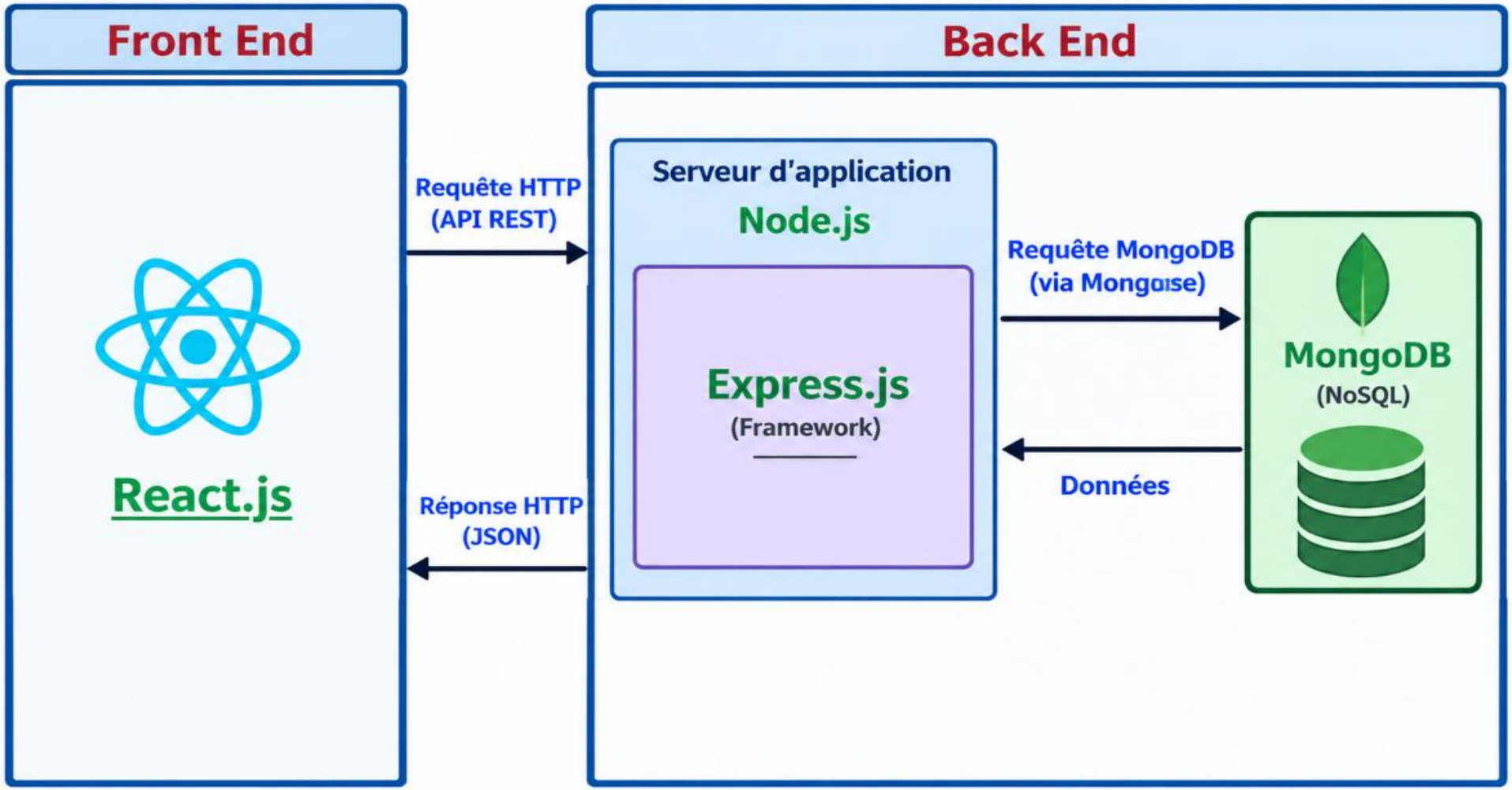
Architecture MVC

1.2 Architecture des applications Web

Architecture Full Stack



Architecture Full Stack JS



MongoDB : Défin

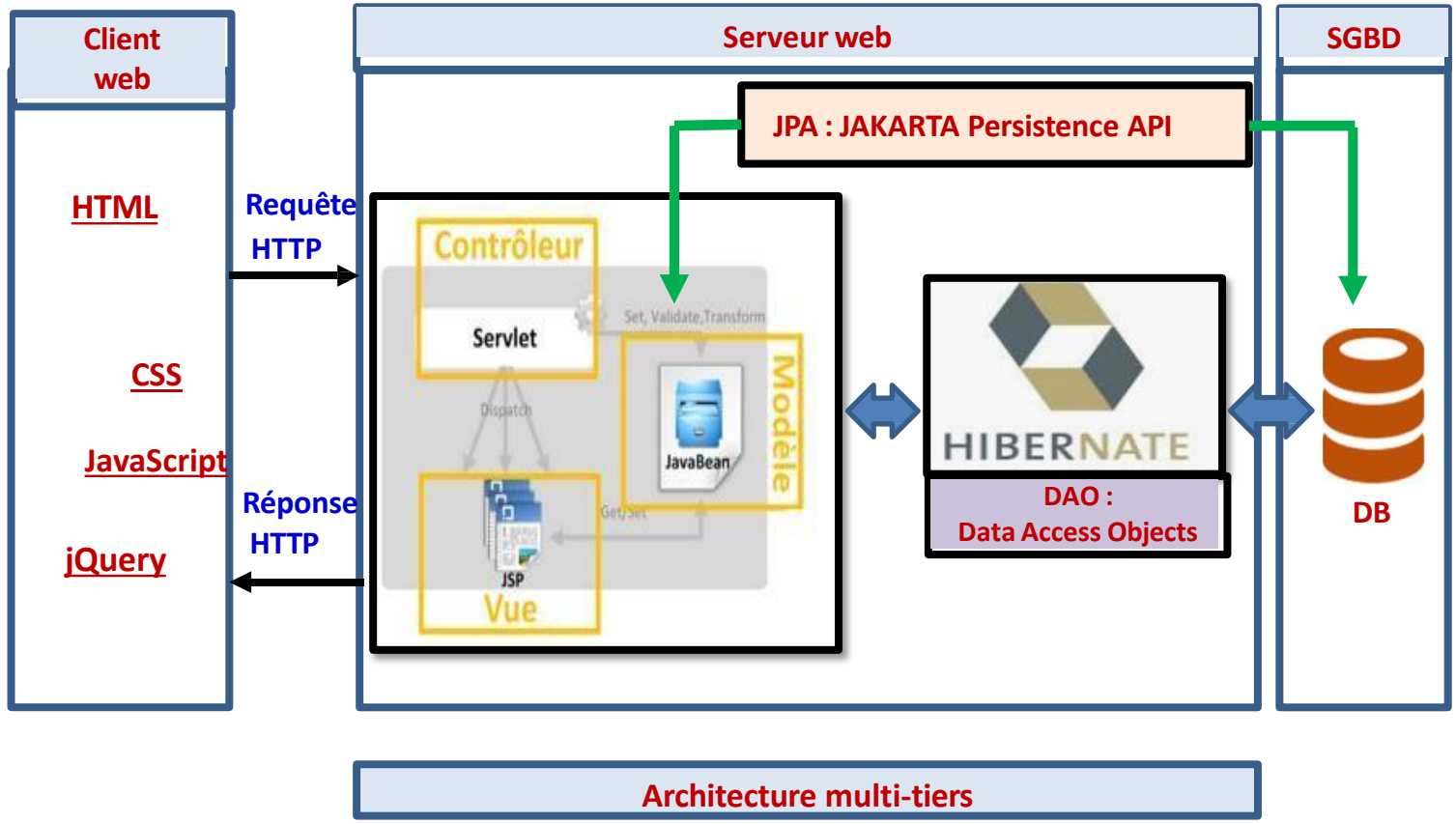


- MongoDB est une base de données NoSQL orientée document.
- Elle se distingue des bases de données relationnelles sa flexibilité et ses performances.
- Contrairement à une base de données relationnelle SQL traditionnelle, MongoDB ne repose sur les tables (des lignes et des colonnes).
- Les données sont stockées sous forme de collections (des

Exemple d’un document

_id: ObjectId('635c3d72dc5eb7442bedbf5d')
Nom: "ABIDI"
Prenom: "Amira"
email: "abidi.Amira@gmail.com"
mdp: "$2a$10$lUCfeVHHCZo5KFmBCpzYfe9XpXG/pkvi5j1m4l2K5qLFXm7lYQCcu"
tel: 25523485
sexe: "Femme"
adresse : "11 Rue de liberté Béja"
cp: "9030"

Architecture JAKARTA EE



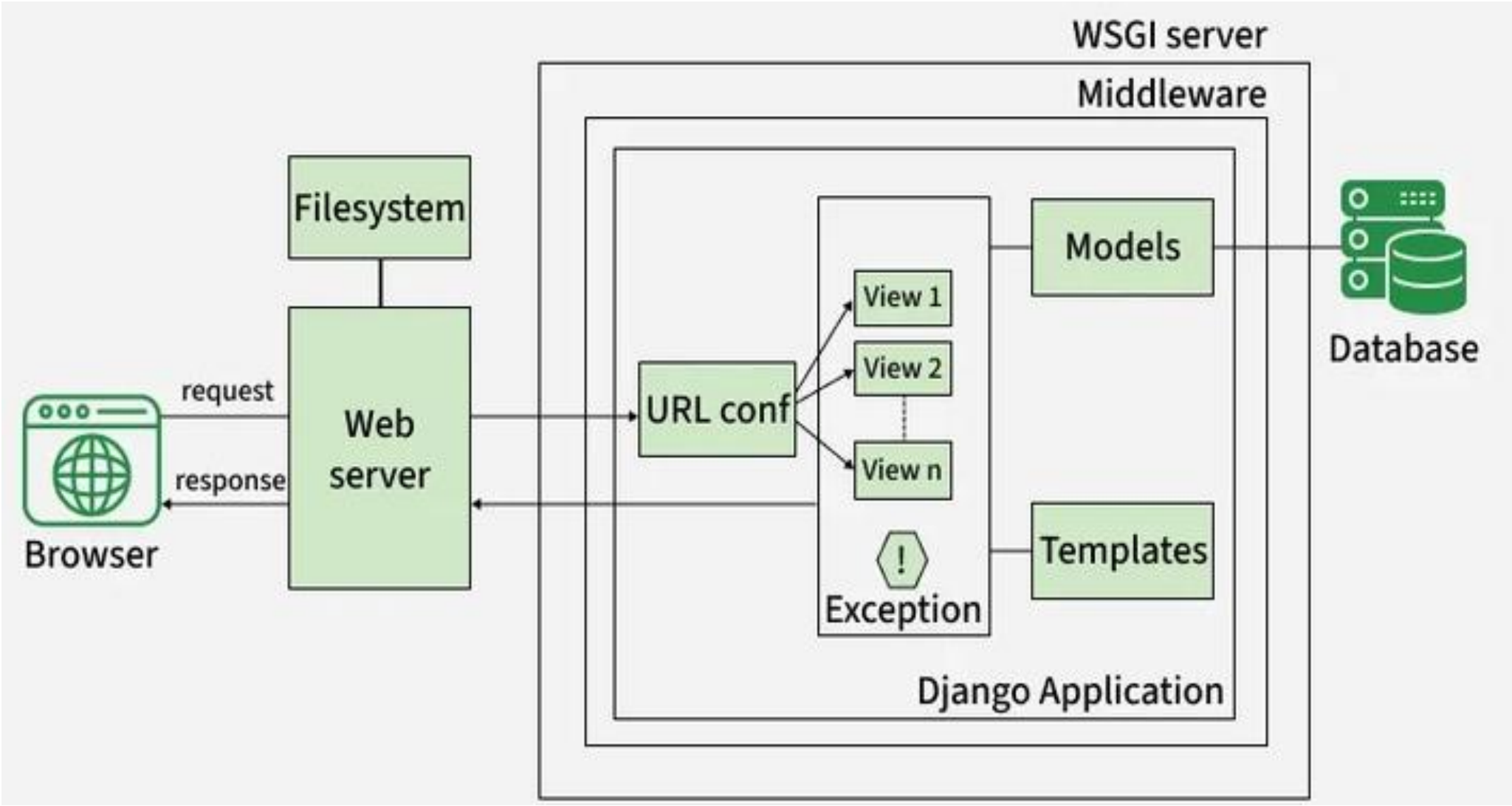
Architecture JEE

- **Servlet :** Une servlet est une classe Java qui s'exécute côté serveur. Elle permet de gérer des requêtes HTTP du client.
- **JSP (Java Server Pages) :** permet d'introduire du code Java (dans des tags JAVA/ balises prédéfinis JAVA) à l'intérieur d'une page HTML.
- **EJB (Entreprise Java Beans) :** les EJB sont des classes Java sérialisable. Un Bean est un objet crée à partir d'un EJB.
- **JPA (Java Persistance API) :** permet le mapping Objet/Relationnel . Il consiste à réaliser la correspondance entre le modèle de données relationnel et le modèle objets Java

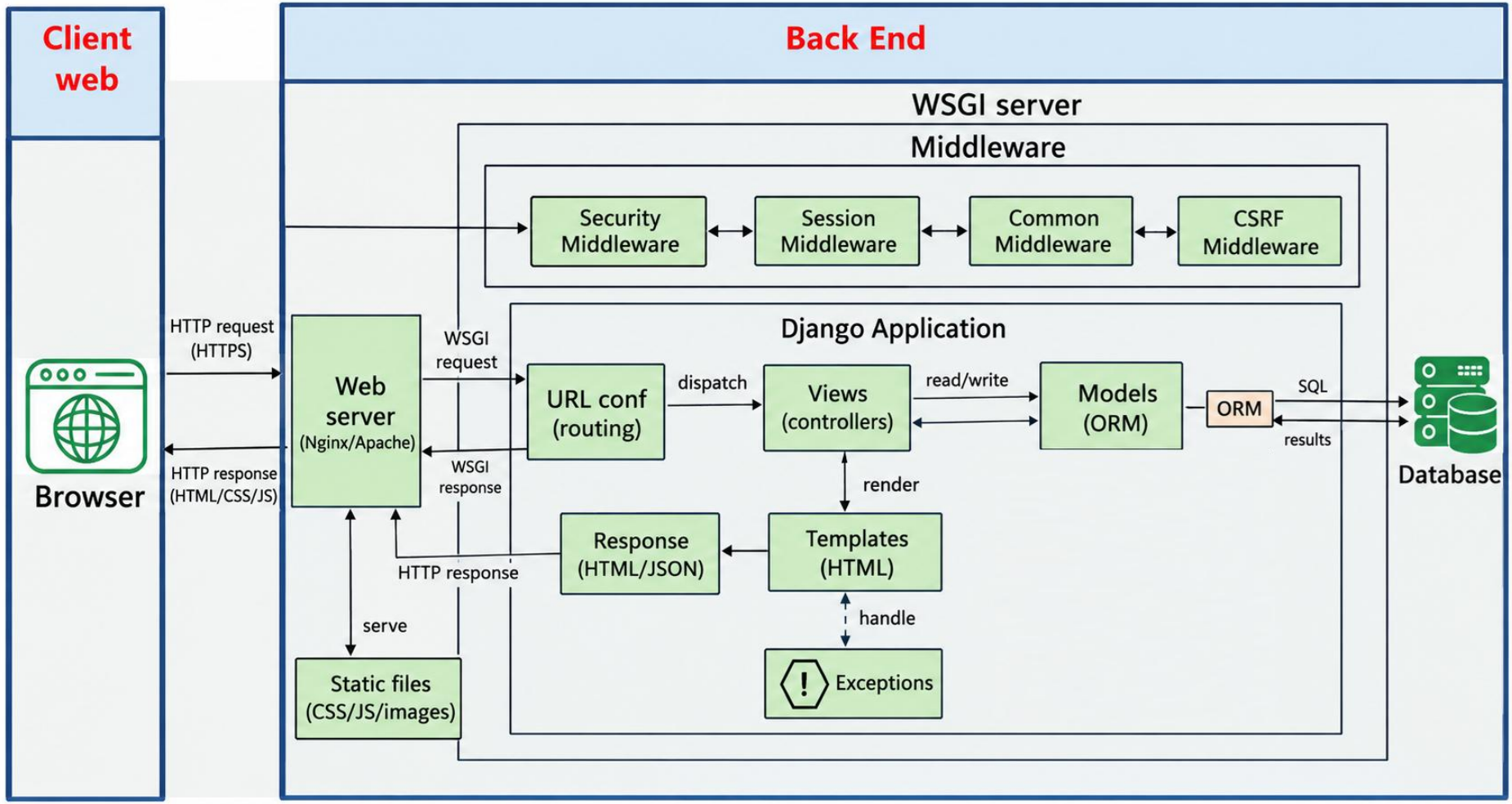
Architecture JEE

- **Servlet :** Hibernate est un framework qui **gère la persistance des objets en base de données relationnelle.**
- **DAO (Data Access Objects) :** Les DAO sont utilisés pour la connexion et la communication avec **notre base de données.** Les DAO sont utilisés lorsque
  - les opérations CRUD (Create, Read, Update, Delete) **sont utilisées dans notre code Java.**

Architecture fonctionnelle Django



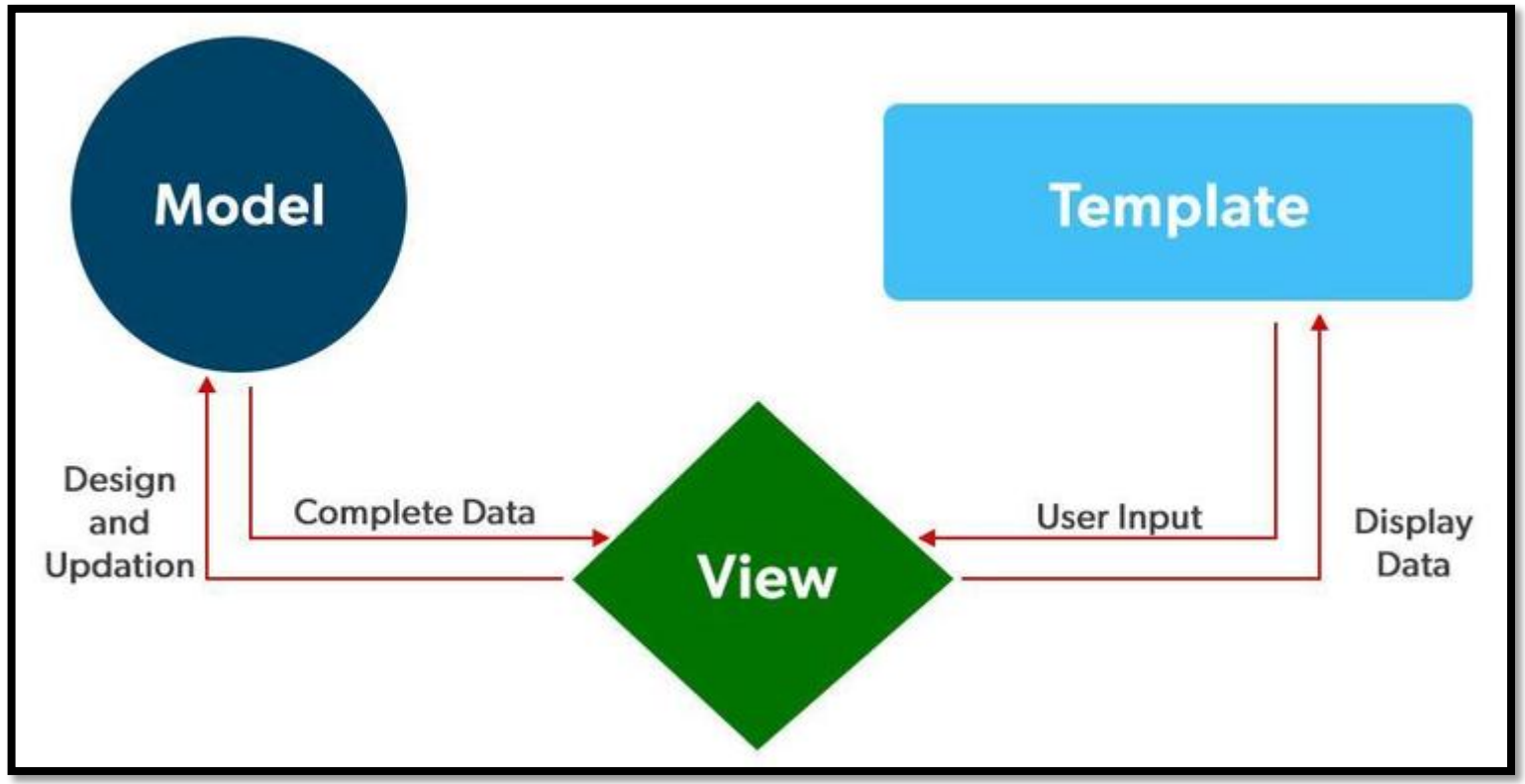
Architecture fonctionnelle Django



*** ORM : Object Relational Mapping**

Architecture MVT Django

- **L'architecture MVT (Modèle-Vue-Template) est le modèle de conception utilisé par le framework Python Django pour séparer la gestion des données (M), la logique métier (V) et l'interface utilisateur (T).**



Architecture MVT Django

Les 3 composants de l'architecture MVT :

- **Modèle (Model) : Il gère les données** de votre application. Écrit **sous forme de classes Python**, il **communique avec la base de données grâce** à **l'ORM (Object-Relational Mapping)** de Django.

- **Vue (View) : Elle contient la logique métier et les traitements.** Elle **reçoit la requête HTTP de l'utilisateur** (**transmise par le système d'URL**), **interagit avec le Modèle pour récupérer ou modifier les données, puis renvoie une réponse.**

- **Template : Il s'occupe de la présentation et de l'affichage des données à l'écran.** C'est **le fichier HTML enrichi par le moteur de templates de Django** qui est renvoyé à l'internaute.

1.3 La Programmation Orientée Objet avec Python

Concepts de base de la POO avec Python

a. Le concept de classe
b. Le concept d'objet (d'instance)
c. L'héritage ,
d. l'encapsulation et le polymorphisme
e. Les classes abstraites
f. Le interfaces

a. Le concept de classe

- Une classe est **un ensemble d'attributs** et **de méthodes regroupées dans une même entité.**
- Une classe est un **modèle de construction des objets**.
- **Instancier** une classe **consiste à créer un objet en utilisant le modèle de la classe.**
- Pour **utiliser une classe** il faut en **créer une ou plusieurs instance(s) (objet(s)) de la classe**.

a. Le concept de classe : exemple

```python
class Personne:
    # Attributs
    nom = None
    prenom = None
    age = None

    # Constructeur de la classe
    def __init__(self, nom, prenom, age):
        self.nom = nom
        self.prenom = prenom
        self.age = age

    # Méthode
    def afficherInfos(self):
        print("Nom : ", self.nom, ", Prenom : ", self.prenom, ", age : ", self.age)
```

b. Le concept d'objet (instance)

Instanciation : création d'objet

- Une classe peut être réutilisée pour instancier plusieurs objets.
- Une instance de classe = Un Objet

```python
class Personne:
    # Attributs
    nom = None
    prenom = None
    age = None

    # Constructeur de la classe
    def __init__(self, nom, prenom, age):
        self.nom = nom
        self.prenom = prenom
        self.age = age

    # Méthode
    def afficherInfos(self):
        print("Nom : ", self.nom, ", Prenom : ", self.prenom, ", age : ", self.age)
```

instanciation

Objets ou instances



Nom : YOUSFI
Prenom : Fares
Age : 24



Nom : ALOUI
Prenom : Kais
Age : 26



Nom : ABIDI
Prenom : Rania
Age : 21

Création d'un objet (instancier une classe)

1. créer un objet en appelant le constructeur de la classe (le constructeur porte toujours le même nom que la classe)

2. Affecter la référence (l'adresse mémoire) de cet objet dans une variable.

- Exemples :

```
P1 = Personne("ABIDI","Rania",21);

P  =  Point('A', 3);
```

La variable self (objet courant)

self est une variable qui sert à **référencer dans une méthode l'instance de l'objet en cours d'utilisation (objet courant)**.

- **Exemple :**

Méthode

```
# Méthode
def afficherInfos(self):
    print("Nom : ", self.nom, ", Prenom : ", self.prenom, ", age : ", self.age)
```

Programme Principal

```
# Création d'un objet de la classe Personne
P1 = Personne("ABIDI", "Rania", 21)
# Appel de la méthode afficher
P1.afficherInfos()
```

Création d'un objet (Etape 1)

```
P1 = Personne("ABIDI","Rania",21);
```

"ABIDI" "Rania" 21

```
def __init__(self, nom, prenom, age):
self.nom = nom
self.prenom = prenom
self.age = age
```

Création d'un objet (Etape 2)

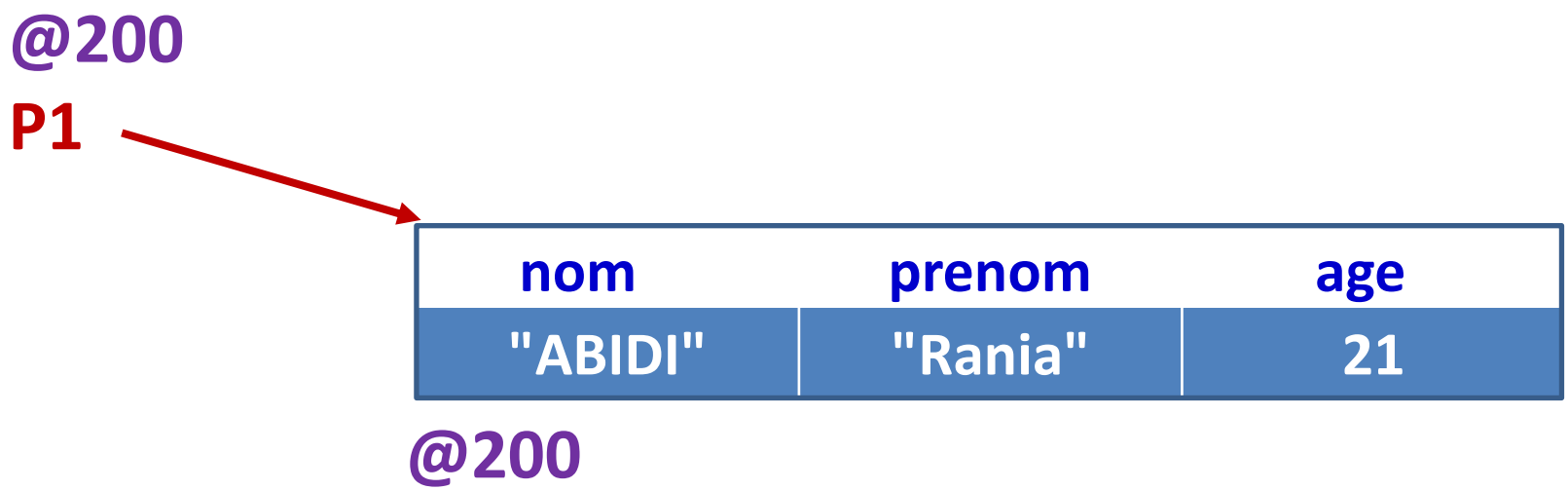
```
P1 = Personne("ABIDI","Rania",21);
```

"ABIDI" "Rania" 21

```
def __init__(P1, nom, prenom, age):
P1.nom = nom
P1.prenom = prenom
P1.age = age
```

Création d'un objet (Etape 3)

```
P1 =  Personne("ABIDI","Rania",21);
```



Création d'un objet : exemple

main.py

```python
class Personne:
    # Attributs
    nom = None
    prenom = None
    age = None

    # Constructeur de la classe
    def __init__(self, nom, prenom, age):
        self.nom = nom
        self.prenom = prenom
        self.age = age

    # Méthode
    def afficherInfos(self):
        print("Nom : ", self.nom, ", Prenom : ", self.prenom, ", age : ", self.age)

# Création d'un objet de la classe Personne
P1 = Personne("ABIDI", "Rania", 21)
# Appel de la méthode afficher
P1.afficherInfos()
```

```
Nom :  ABIDI , Prenom :  Rania , age :  21
```

Durée de vie d'un objet

- **La durée de vie d'un objet passe par 3 étapes :**

1. **La création** de l'objet (**l'instanciation**),
2. **l'utilisation** de l'objet **en appelant ces méthodes**,
3. **La suppression** de l'objet : elle est **automatique.**

- **L'interpréteur Python libère automatiquement la mémoire des objets inutilisés par le programme pour la rendre disponible pour d'autres objets.**

Constructeurs

- Lors de la **création d'un objet** (**instanciation d'une classe**), on fait **automatiquement appel** à **une méthode** appelée **constructeur**.

- Cette méthode permet **généralement** **d'affecter des valeurs aux variables** définies dans la classe.

- **Une classe peut posséder 0, un constructeur.**

- **On ne peut pas avoir plusieurs constructeurs dans une classe, mais on peut utiliser la valeur par défaut des paramètres pour arriver à la même fin.**

Constructeurs

- Le constructeur se définit dans une classe comme une fonction avec deux particularités :
  - le nom de la fonction doit être __init__ ;
  - la fonction doit accepter au moins un paramètre, dont le nom doit être self, et qui doit être le premier paramètre.
- Constructeur par défaut:
  - Un constructeur sans aucun paramètre est appelé un constructeur par défaut.
  - Si nous ne créons pas de constructeur, la classe appellera automatiquement le constructeur par défaut crée par l'interpréteur lorsqu'un objet est créé.

Destructeurs

- **Un destructeur** est une méthode qui permet de **libérer l'espace mémoire occupé par l'objet**.

- **La libération de la mémoire** (variables, constantes et objets) est **prise en charge** par **le ramasse-miettes**.

- La méthode **__del __ ()** est **une méthode appelée destructeur en Python.**

Syntaxe

```
1  def __del__(self):
2      # actions
```

Accesseurs

- **Définition:**

> Un **accesseur** est **une méthode** qui **permet** d'**accéder** en **lecture** ou en **écriture** à **la valeur d'un attribut**.

- Par convention, **le rôle des accesseurs se limite à**:

  - **La lecture: retourner la valeur de l'attribut concerné,**
    ou
  - **L' écriture : modifier la valeur de l'attribut concerné.**

- Les accesseurs **portent le nom** de **get (pour la lecture)** ou **set (pour l'écriture) suivi du nom de l'attribut concerné**.

Le constructeur : exemple

main.py

```python
class Personne:
    # Attributs
    nom = None
    prenom = None
    age = None
    email = None

    # Constructeurs
    def __init__(self, nom, prenom, age, email=None):
        self.nom = nom
        self.prenom = prenom
        self.age = age
        self.email = email

    # Méthode
    def afficherInfos(self):
        if self.email is None:
            print("Nom : ", self.nom, ", Prenom : ", self.prenom, ", age : ", self.age)
        else:
            print("Nom : ", self.nom, ", Prenom : ", self.prenom, ", age : ", self.age, ", email : ", self.email)

# Création de 2 objets de la classe Personne
P1 = Personne("ABIDI", "Rania", 21)
P2 = Personne("ALOUI", "Amir", 23,"amir.aloui@gmail.com")
# Appel de la méthode afficher
P1.afficherInfos()
P2.afficherInfos()
```

```
Nom :  ABIDI , Prenom :  Rania , age :  21
Nom :  ALOUI , Prenom :  Amir , age :  23 , email :  amir.aloui@gmail.com
```

Le constructeur : exemple (suite)

main.py

```python
class Personne:
    # Attributs
    nom = None
    prenom = None
    age = None

    # Accesseurs
    def setNom(self, nom):
        self.nom = nom

    def getNom(self):
        return self.nom

    def setPrenom(self, prenom):
        self.prenom = prenom

    def getPrenom(self):
        return self.prenom

    def setAge(self, age):
        self.age = age

    def getAge(self):
        return self.age
```

Le constructeur : exemple (suite)

```
26          # Constructeurs
27          def __init__(self, nom, prenom, age):
28              self.nom = nom
29              self.prenom = prenom
30              self.age = age
31
32          # Méthode
33          def afficherInfos(self):
34              print("Nom : ", self.nom, ", Prenom : ", self.prenom, ", age : ", self.age)
35
36      # Création d'un objet de la classe Personne
37      P1 = Personne("ABIDI", "Rania", 21)
38      P1.setNom("ABIDI ALOUI")
39      P1.setAge(23)
40      P1.afficherInfos()
41      print("Le prénom de P1 = ", P1.getPrenom())
```

```
Nom :  ABIDI ALOUI , Prenom :  Rania , age :  23
Le prénom de P1 =  Rania
```

Le constructeur par défaut : exemple 1

main.py

```python
class Personne:
    # Attributs
    nom = None
    prenom = None
    age = None

    # Accesseurs
    def setNom(self, nom):
        self.nom = nom

    def getNom(self):
        return self.nom

    def setPrenom(self, prenom):
        self.prenom = prenom

    def getPrenom(self):
        return self.prenom

    def setAge(self, age):
        self.age = age

    def getAge(self):
        return self.age
```

Le constructeur par défaut : exemple 1

```
25      # Méthode
26      def afficherInfos(self):
27          print("Nom : ", self.nom, ", Prenom : ", self.prenom, ", age : ", self.age)
28
29  # Création d'un objet de la classe Personne
30  P1 = Personne()
31  P1.afficherInfos()
```

Exécution

```
Nom :  None , Prenom :  None , age :  None
```

Le constructeur par défaut : exemple 2

main.py

```
1  class Personne:
2      nom = None
3      prenom = None
4      age = None
5
6      # Accesseurs
7      def setNom(self, nom):
8          self.nom = nom
9
10     def getNom(self):
11         return self.nom
12
13     def setPrenom(self, prenom):
14         self.prenom = prenom
15
16     def getPrenom(self):
17         return self.prenom
18
19     def setAge(self, age):
20         self.age = age
21
22     def getAge(self):
23         return self.age
```

Le constructeur par défaut : exemple 2

```
25      # Méthode
26      def afficherInfos(self):
27          print("Nom : ", self.nom, ", Prenom : ", self.prenom, ", age : ", self.age)
28
29  # Création d'un objet de la classe Personne
30  P1 = Personne()
31  P1.setNom("ABIDI")
32  P1.setPrenom("Rania")
33  P1.setAge(21)
34  P1.afficherInfos()
```

Exécution

```
Nom :  ABIDI , Prenom :  Rania , age :  21
```

Variable de classe

- **Une variable de classe est la même pour tous les objets de ta classe.**
- Elles ne sont **définies qu'une seule fois quelque soit le nombre d'objets instanciés** de la classe.
- **Si on modifier la valeur de la variable de classe**, cette valeur **sera modifié pour tout les objets de la classe.**
- **L'appartenance des variables de classe à une classe entière** et **non à un objet spécifique** **permet de remplacer le nom de la variable par le nom de la classe.**

Variable de classe

personne.py

```python
class Personne:
    # Attributs
    nom = None
    prenom = None
    age = None

    anneeCourante = 2026  # Variable de classe

    # Constructeur
    def __init__(self, nom, prenom, age):
        self.nom = nom
        self.prenom = prenom
        self.age = age

    # Méthode
    def afficherInfos(self):
        print("Nom : ", self.nom, ", Prenom : ", self.prenom, ", age : ", self.age)
```

Variable de classe

main.py

```
1  from personne import Personne
2
3  # Création de 2 objets de la classe Personne
4  P1 = Personne("ABIDI", "Rania", 21)
5  P2 = Personne("HAJJI", "Rami", 23)
6  print("Année courante de P1 : ", P1.anneeCourante)
7  print("Année courante de P2 : ", P2.anneeCourante)
8  Personne.anneeCourante = 2027
9  print("Année courante de P1 : ", P1.anneeCourante)
10 print("Année courante de P2 : ", P2.anneeCourante)
```

Méthode de classe

- Une méthode de classe est une méthode statique qu'elle peut être appelée sans instancier sa classe.

- Syntaxe : Classe.methode() au lieu de Objet.méthode())

- En général, une classe possédant des méthodes statiques n'est pas conçu pour être instanciée (ne peut pas être appelée par des objets).

- Une méthode statique ne doit pas contenir le mot clé self.

Méthode de classe

main.py

```
1   class Calcul:
2
3       @staticmethod
4       def somme(a, b):
5           return a + b
6
7   # Programme principal
8   x, y = 10, 20
9   z = Calcul.somme(x, y)
10  print (x, "+", y, "=", z)
```

```
10 + 20 = 30
```

c. L'héritage et le polymorphisme

Définition

- **L'héritage** est **l'un des principaux concepts de la P.O.O.**

- En POO, il est possible de créer des **classes drivant d'une autre** : On **appelle cela l'héritage.**
- On appelle **classe mère, la classe de base.**
- On appelle **classe fille, celle qui dérive (hérite) de la classe mère.**

- **Pour indiquer qu'une classe hérite d'une autre**, il faut **utiliser cette syntaxe :**

```
class Fille (Mere(s)):
    //ici le code spécifique de la classe fille
```

Objectifs de l'héritage

- On souhaite **décrire une seule fois** le **même traitement** lorsqu'il **s'applique à plusieurs classes**.

- **Solution:** **créer une classe générique.**

- **Une classe fille** peut avoir **une ou plusieurs classe(s) mère().**

- **Python autorise l'héritage multiple.**

- **Une classe mère** peut avoir **une ou plusieurs classes filles.**

- Toutes **les classes spécifiques (classes filles)**, **héritent de la (les) classe(s) générique(s) (classe(s) mère(s)).**

La classe Object

- **Toutes les classes python (les classes prédéfinies et les classes créées par les programmeurs)** hérite **indirectement** ou **directement** de **la classe object**.

- **Déclaration d'une classe**

```
1  class MaClasse:
2      """
3      coprs de la classe
4      """
```

- **Ou**

```
1  class MaClasse(object):
2      """
3      coprs de la classe
4      """
```

La classe Object

- **Quelques méthodes de la classe object :**

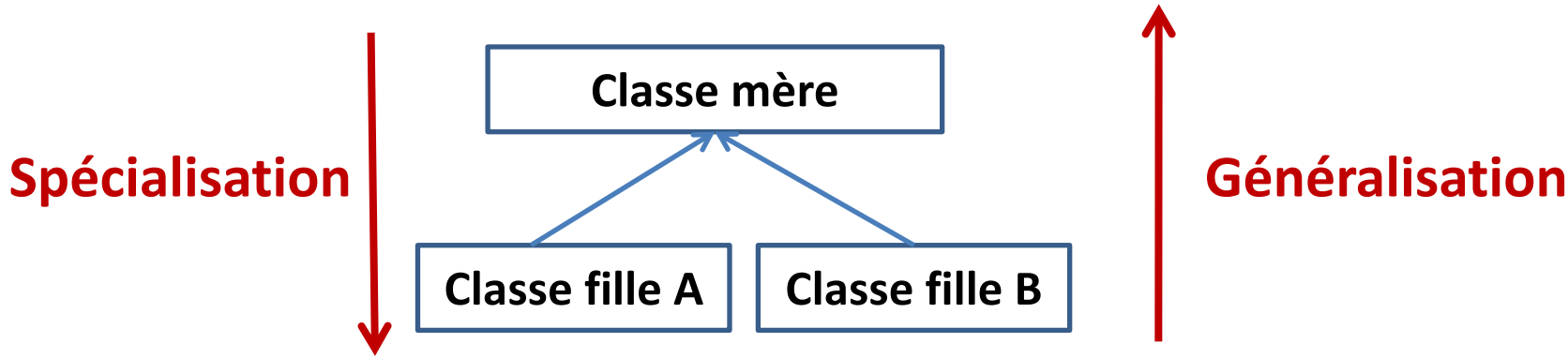
- La méthode **__init__** est là pour **initialiser un objet** non pour **le créer**. La **méthode qui s'en** charge, c'est **__new__**

- La méthode **__new__** de la classe **est appelée et se charge de créer un nouvel objet.**

- La méthode **__str__** est appelée pour **afficher l'objet** (convertir l'objet en chaîne de caractères).

- …

Usage de l'héritage



- **La classe mère (classe générique) contient tous qui est commun entre les classes filles (càd les attributs et les méthodes communes).**

- **Une classe fille** peut contient des **attributs et des méthodes spécifiques à elle.**

Usage de l'héritage

- **Une classe fille hérite les attributs** et **les méthodes** de sa(ses) classe(s) mère(s) **(sauf ceux qui sont privés)**.

- **On n'a pas besoin de les réécrire** pour la classe fille.

- **Une classe fille peut contenir des attributs et des méthodes spécifiques à elle**.

- Une classe fille peut **redéfinir une méthode de sa classe mère** (**de même signature, càd de même entête**).

- **Le constructeur de la classe fille doit toujours commencer par appeler le constructeur de sa classe mère.**

La méthode super() : redéfinition de constructeur

- **La méthode super()** permet d'**appeler le constructeur de la classe mère.**

- **Appeler le constructeur de la classe mère garantit qu'on peut initialiser les paramètres (arguments) de la classe mère.**

- **On lui passe** les **valeurs des paramètres** .

La méthode super() : redéfinition de méthode

- **Super().nomMethode()** permet d'appeler la méthode **nomMethode()** hérité de la classe mère.

⇨ **Réutiliser le code de la méthode héritée.**

Exemple : super().afficherInfos() **# Appeler la méthode**
afficherInfos() de la classe mère.

Redéfinition d’un attribut

- **Redéfinir un attribut est très simple, puisqu'il suffit de redéfinir sa valeur au sein de la classe fille.**

```
class MaClasseFille(MaClasseMere):
    def __init__(self):
        super().__init__(self)
        self.attribut_classe_mere = 'nouvelle valeur'
```

L’encapsulation

- **Définition:**

- Le terme **encapsulation** désigne le principe consistant à **cacher l'information contenue dans un objet.**

- L'objet est vu de l'extérieur comme une **boîte noire** ayant certaines **propriétés (attributs)** et ayant **un comportement (méthodes)**.

- La **manière dont le comportement a été implémenté** est **cachée** aux **utilisateurs** de l'objet.

L’encapsulation

- **L'encapsulation nécessite la spécification de parties publiques et privées de l'objets:**

  - **Éléments publics :** correspond à la **partie visible de l'objet depuis l'extérieur**. c'est un ensemble d’attributs et de méthodes utilisables par d'autres objets de l’environnement.

  - **Éléments privées : correspond à la partie non visible de l'objet**. Il est constitué des éléments de l'objet visibles **uniquement de l'intérieur de l'objet**.

L’encapsulation

- Python ne prend pas en charge **l’encapsulation**.

- Pour **rendre les attributs et les méthodes privés** en Python, **préfixez le nom avec un double trait de soulignement.**

L’encapsulation : Exemple_v1

personne.py

```python
class Personne:
    # Attributs
    nom = None
    prenom = None
    age = None

    # Constructeur de la classe
    def __init__(self, nom, prenom, age):
        self.nom = nom
        self.prenom = prenom
        self.age = age

    # Accesseurs
    def setAge(self, age):
        self.age = age

    def getAge(self):
        return self.age

    # méthode
    def afficherInfos(self):
        print("Nom : ", self.nom, ", Prenom : ", self.prenom, ", Age : ", self.age)
```

L’encapsulation : Exemple_v1

main.py

```
1  from personne import Personne
2
3  # Programme Principal
4  P1 = Personne("ABIDI", "Ranim", 22)
5  P1.setAge(23)
6  P1.afficherInfos()
```

```
Nom :  ABIDI , Prenom :  Ranim , Age :  23
```

L’encapsulation : Exemple_v2

personne.py

```python
class Personne:
    # Attributs
    __nom = None
    __prenom = None
    __age = None

    # Constructeur de la classe
    def __init__(self, nom, prenom, age):
        self.nom = nom
        self.prenom = prenom
        self.age = age

    # Accesseurs
    def __setAge(self, age):
        self.age = age

    def __getAge(self):
        return self.age

    # méthode
    def __afficherInfos(self):
        print("Nom : ", self.nom, ", Prenom : ", self.prenom, ", Age : ", self.age)
```

L’encapsulation : Exemple_v2

main.py

```python
from personne import Personne

# Programme Principal
P1 = Personne("ABIDI", "Ranim", 22)
P1.__setAge(23)
P1.__afficherInfos()
```

```
Traceback (most recent call last):
  File "C:\Users\nabil\PycharmProjects\MyProject\main.py", line 5, in <module>
    P1.__setAge(23)
    ^^^^^^^^^^
AttributeError: 'Personne' object has no attribute '__setAge'
```

L'encapsulation : Exemple_v3 (main.py)

main.py

```python
class Personne:
    # Attributs
    __nom = None
    __prenom = None
    __age = None

    # Constructeur de la classe
    def __init__(self, nom, prenom, age):
        self.nom = nom
        self.prenom = prenom
        self.age = age

    # Accesseurs
    def __setAge(self, age):
        self.age = age

    def __getAge(self):
        return self.age

    # méthode
    def __afficherInfos(self):
        print("Nom : ", self.nom, ", Prenom : ", self.prenom, ", Age : ", self.age)
```

L’encapsulation : Exemple_v3 (main.py)

```
23
24  # Programme Principal
25  P1 = Personne("ABIDI", "Ranim", 22)
26  P1._Personne__setAge(23)
27  P1._Personne__afficherInfos()
```

```
Nom :  ABIDI , Prenom :  Ranim , Age :  23
```

Le polymorphisme

Polymorphisme = poly + morphisme

plusieurs *forme*

❑ **Définition:**

Le polymorphisme est la **capacité d'un objet à posséder plusieurs formes**.

❑ **On distingue 2 types de polymorphisme :**

A - La surcharge de méthode et

B - La redéfinition de méthode .

Le polymorphisme

- **A – La surcharge :**

- **La surcharge de méthode (ou polymorphisme statique) consiste dans le fait qu'une classe peut disposer de plusieurs méthodes ayant le même nom.**

- **Mais avec des paramètres différents ou éventuellement un type de retour différent.**

- **Remarque : Python ne prend pas en charge la surcharge de méthode.**

- **Nous pouvons surcharger les méthodes mais nous ne pouvons utiliser que la dernière méthode définie.**

Le polymorphisme

A – La surcharge de méthode :

main.py

```
1   class Addition:
2
3       @staticmethod
4       def somme(a, b):
5           return a + b
6
7       @staticmethod
8       def somme(a, b, c):
9           return a + b + c
10
11
12
13  # Programme principal
14  x, y, z = 2, 3, 5
15  r = Addition.somme(x, y)   # Méthode de classe
16  print(r)
```

```
Traceback (most recent call last):
  File "C:\Users\nabil\PycharmProjects\MyProject\main.py", line 15, in <module>
    r = Addition.somme(x, y)
        ^^^^^^^^^^^^^^^^^^^^
TypeError: Addition.somme() missing 1 required positional argument: 'c'
```

Le polymorphisme

A – La surcharge de méthode :

main.py

```
1   class Addition:
2
3       @staticmethod
4       def somme(a, b):
5           return a + b
6
7       @staticmethod
8       def somme(a, b, c):
9           return a + b + c
10
11
12
13  # Programme principal
14  x, y, z = 2, 3, 5
15  r = Addition.somme(x, y, z)   # Méthode de classe
16  print(r)
```

```
10
```

Le polymorphisme

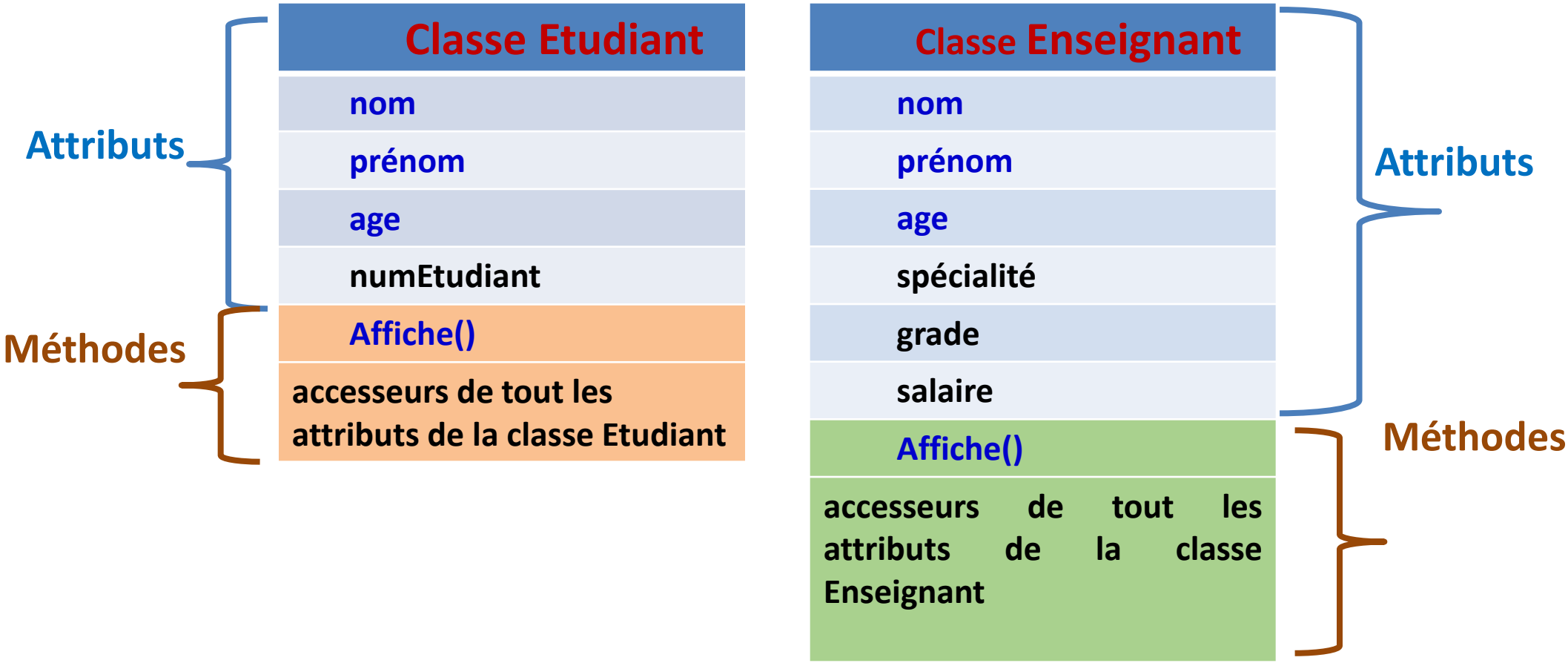
- **B – Redéfinition :**

- **La redéfinition de méthode (ou polymorphisme dynamique) est spécifique aux langages orientés objet.**

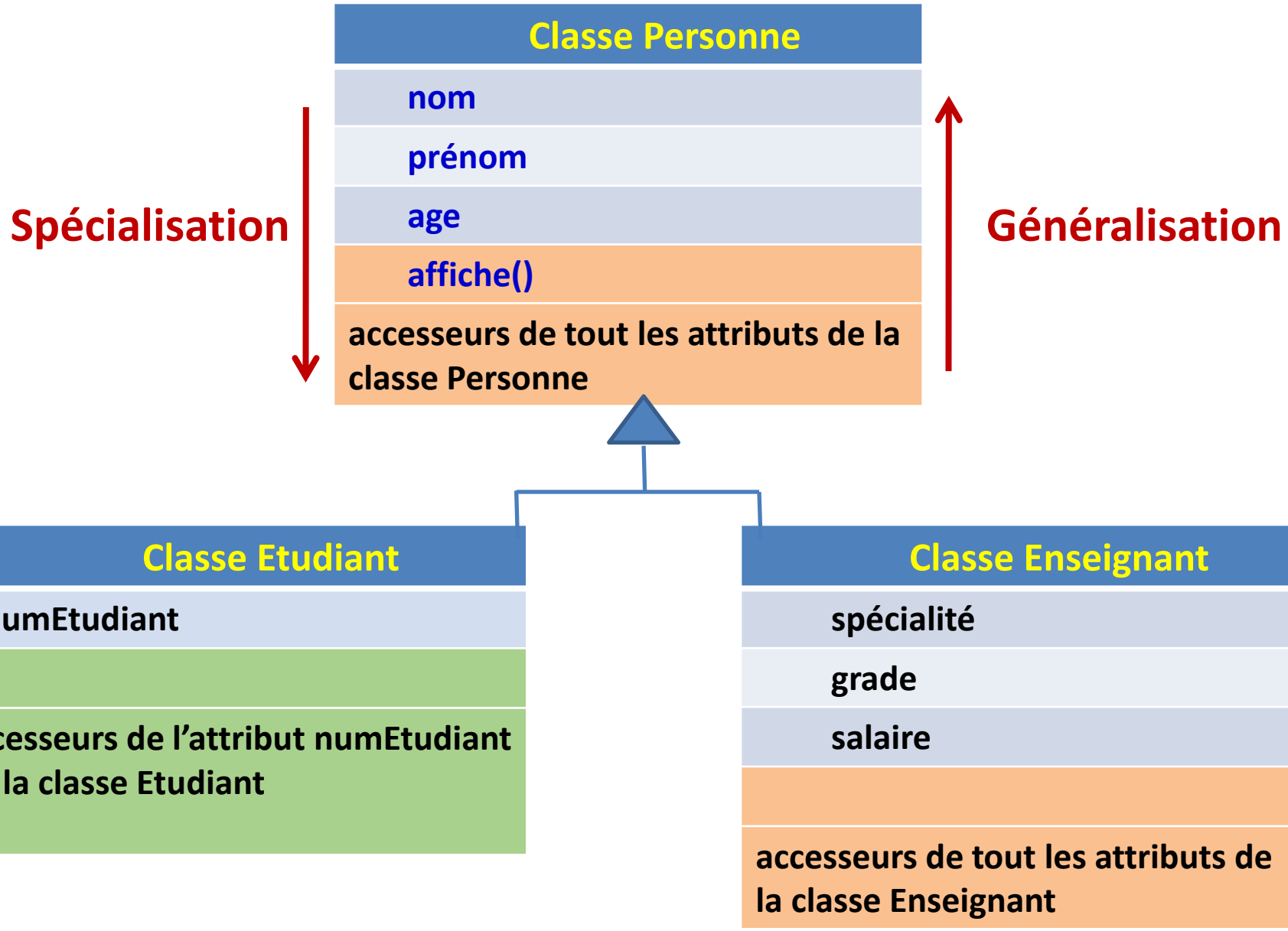
- **La redéfinition** consiste à donner une **nouvelle implémentation (dans une classe fille)** à **une méthode héritée (d'une classe mère) sans changer sa signature :**

  - **Le même nom de méthode;**
  - **Le même type de retour;**
  - **Les mêmes paramètre(s) (nombre et type(e) de paramètre(s)).**

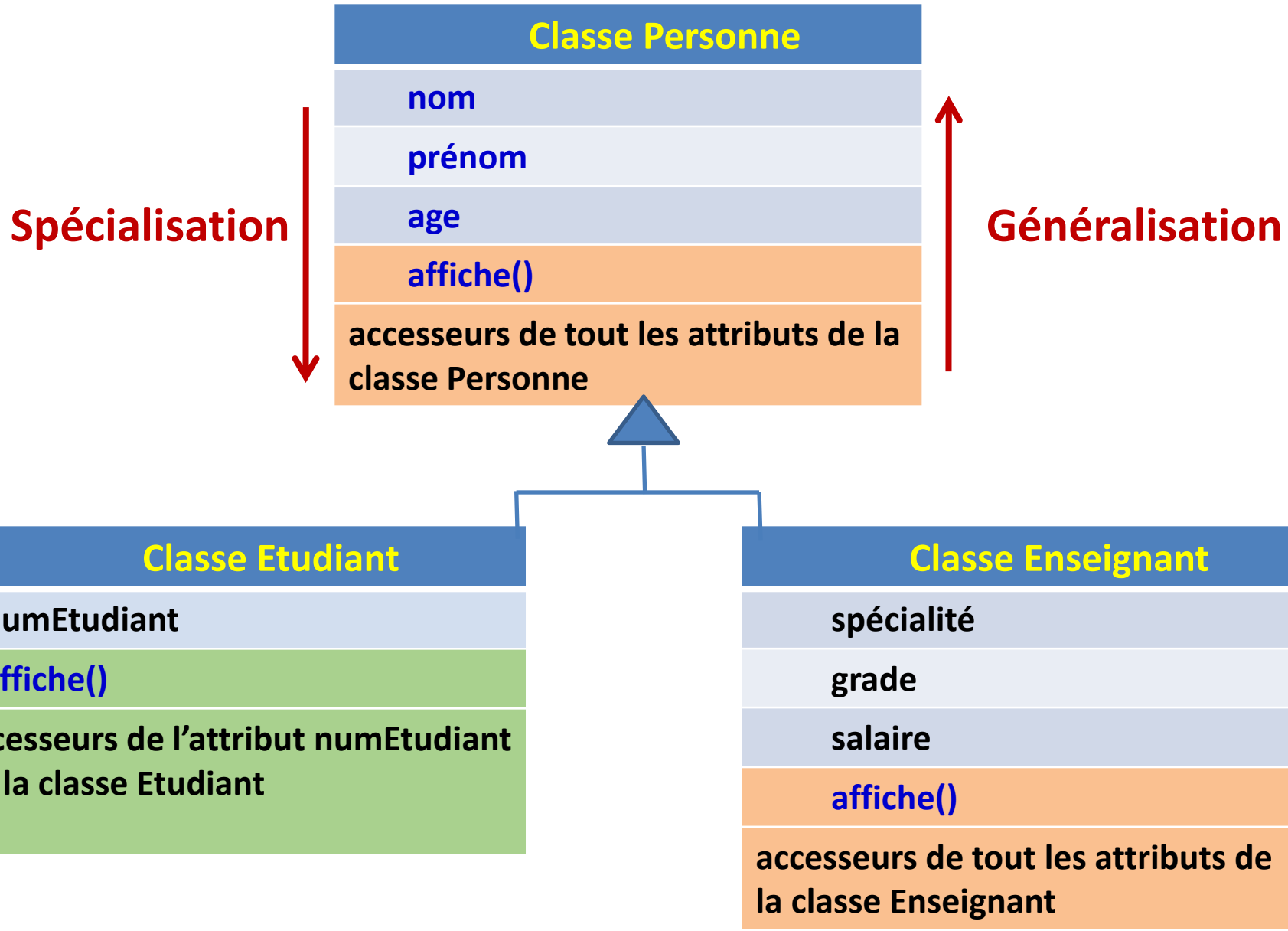
Exemple


Classe Etudiant
nom
prénom
age
numEtudiant
Affiche()
accesseurs de tout les attributs de la classe Etudiant
Attributs
Méthodes
Classe Enseignant
nom
prénom
age
spécialité
grade
salaire
Affiche()
accesseurs de tout les attributs de la classe Enseignant
Attributs
Méthodes

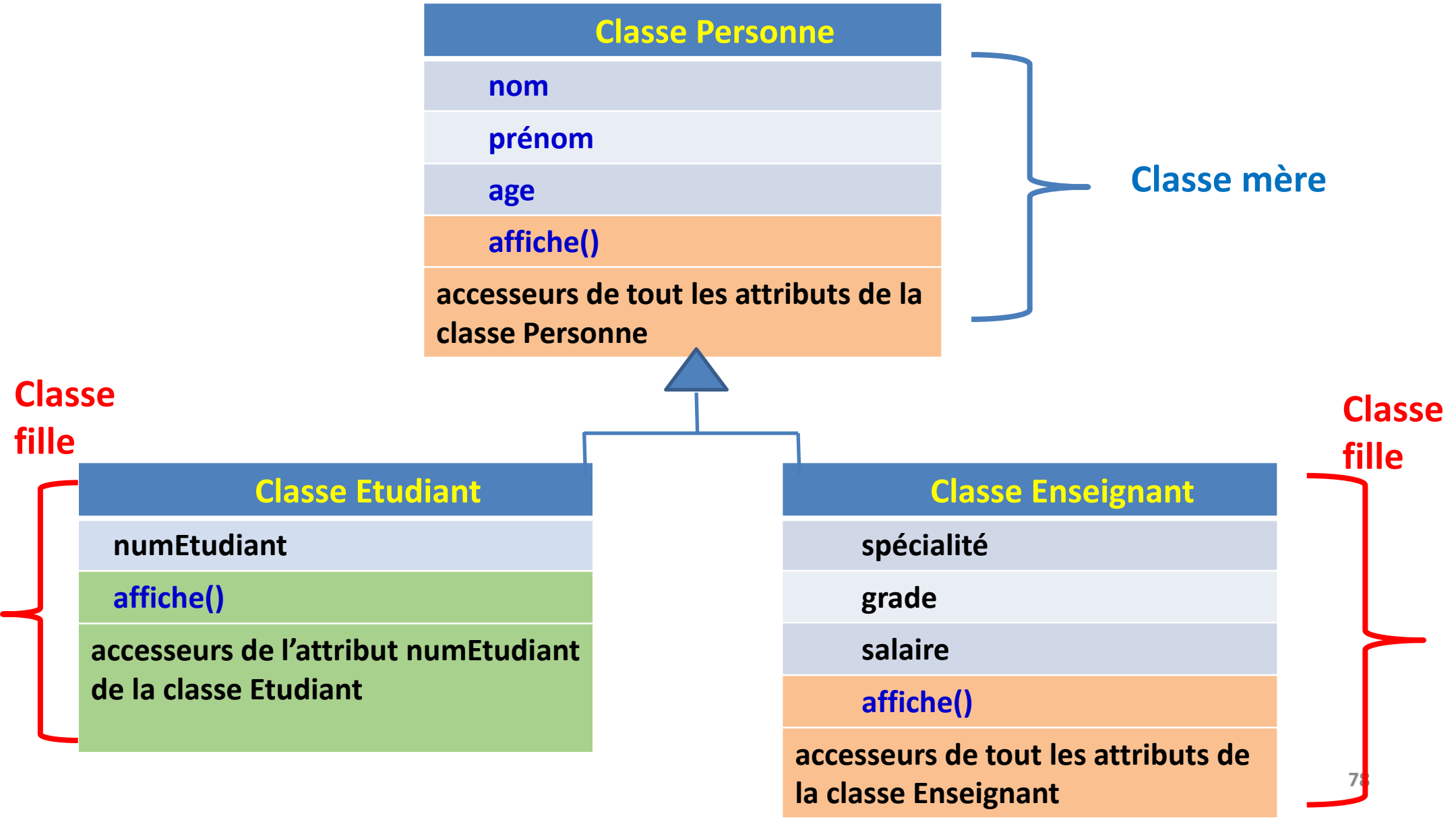
Exemple


Classe Personne
nom
prénom
age
affiche()
accesseurs de tout les attributs de la classe Personne
Spécialisation
Généralisation
Classe Etudiant
numEtudiant
accesseurs de l'attribut numEtudiant de la classe Etudiant
Classe Enseignant
spécialité
grade
salaire
accesseurs de tout les attributs de la classe Enseignant

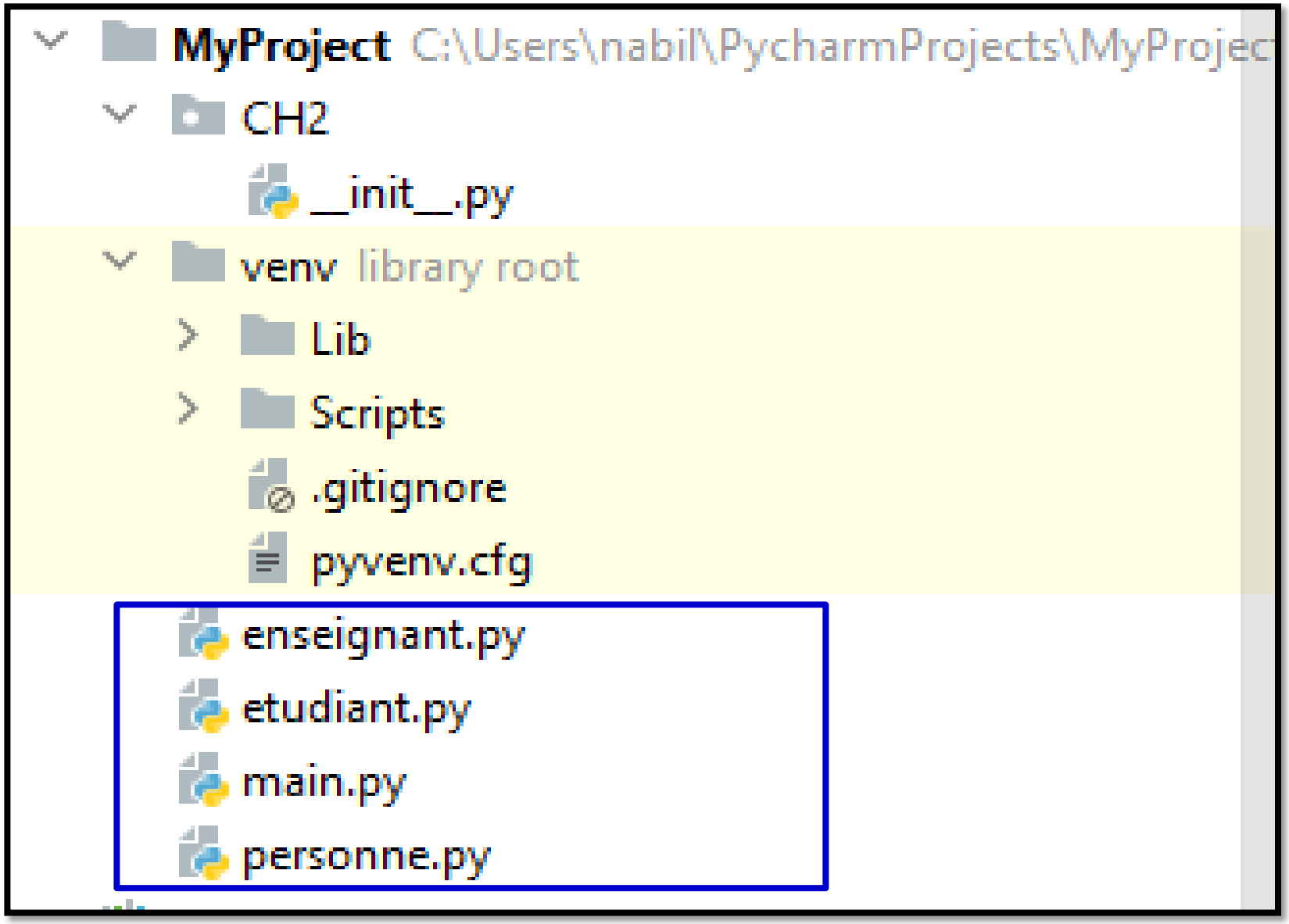
Exemple


Classe Personne
nom
prénom
age
affiche()
accesseurs de tout les attributs de la classe Personne
Spécialisation
Généralisation
Classe Etudiant
numEtudiant
affiche()
accesseurs de l'attribut numEtudiant de la classe Etudiant
Classe Enseignant
spécialité
grade
salaire
affiche()
accesseurs de tout les attributs de la classe Enseignant

Exemple


Classe Personne
nom
prénom
age
affiche()
accesseurs de tout les attributs de la classe Personne
Classe mère
Classe fille
Classe Etudiant
numEtudiant
affiche()
accesseurs de l'attribut numEtudiant de la classe Etudiant
Classe fille
Classe Enseignant
spécialité
grade
salaire
affiche()
accesseurs de tout les attributs de la classe Enseignant

Exemple_v1


MyProject C:\Users\nabil\PycharmProjects\MyProject
CH2
__init__.py
venv library root
Lib
Scripts
.gitignore
pyvenv.cfg
enseignant.py
etudiant.py
main.py
personne.py

Exemple v1 : classe Personne

personne.py

```python
class Personne:
    # Attributs
    nom = None
    prenom = None
    age = None

    # Constructeur de la classe
    def __init__(self, nom, prenom, age):
        self.nom = nom
        self.prenom = prenom
        self.age = age

    # Accesseur de l'attribut age
    def setNom(self, nom):
        self.nom = nom

    def getNom(self):
        return self.nom

    def setPrenom(self, prenom):
        self.prenom = prenom

    def getPrenom(self):
        return self.prenom
```

Exemple_v1 : classe Personne

```
25
26      def setAge(self, age):
27          self.age = age
28
29      def getAge(self):
30          return self.age
31
32      # méthode
33      def affiche(self):
34          print("Nom : ", self.nom, ", Prenom : ", self.prenom, ", age : ", self.age)
35
```

Exemple v1 : classe Etudiant

etudiant.py

```python
from personne import Personne

class Etudiant(Personne):
    numEtd = None

    # Constructeur de la classe
    def __init__(self, nom, prenom, age, numEtd):
        Personne.__init__(self, nom, prenom, age)
        self.numEtd = numEtd

    # Accesseur de l'attribut age
    def setNumEtd(self, numEtd):
        self.numEtd = numEtd

    def getNumEtd(self):
        return self.numEtd

    # Rédefinition de méthode
    def affiche(self):
        Personne.affiche(self)
        print("NumEtd : ", self.numEtd)
```

Exemple_v1 : classe Enseignant

enseignant.py

```python
from personne import Personne

class Enseignant(Personne):
    specialite = None
    grade = None
    salaire = None

    # Constructeur de la classe
    def __init__(self, nom, prenom, age, specialite, grade, salaire):
        Personne.__init__(self, nom, prenom, age)
        self.specialite = specialite
        self.grade = grade
        self.salaire = salaire

    # Accesseurs
    def setSpecialite(self, specialite):
        self.specialite = specialite

    def getSpecialite(self):
        return self.specialite

    def setGrade(self, grade):
        self.grade = grade
```

Exemple v1 : classe Enseignant

```
24
25      def getGrade(self):
26          return self.grade
27
28      def setSalaire(self, salaire):
29          self.salaire = salaire
30
31      def getSalaire(self):
32          return self.salaire
33
34      # Rédefinition de méthode
35      def affiche(self):
36          Personne.affiche(self)
37          print(", specialité : ", self.specialite,", grade : ", self.grade,", salaire : ", self.salaire)
38
```

Exemple_v1 : main.py

main.py

```python
from etudiant import Etudiant
from enseignant import Enseignant

# Programme Principal
Et1 = Etudiant("ALOUI", "kais", 30, 587)
En1 = Enseignant("KHEMIRI", "Amira", 29, "Informatique", "MA", 2500)
En1.setSalaire(2600)
Et1.affiche()
En1.affiche()
```

```
Nom :  ALOUI , Prenom :  kais , age :  30
NumEtd :  587
Nom :  KHEMIRI , Prenom :  Amira , age :  29
, specialité :  Informatique , grade :  MA , salaire :  2600
```

Exemple v2 : classe Personne

personne.py

```python
class Personne:
    # Attributs
    nom = None
    prenom = None
    age = None

    # Constructeur de la classe
    def __init__(self, nom, prenom, age):
        self.nom = nom
        self.prenom = prenom
        self.age = age

    # Accesseur de l'attribut age
    def setNom(self, nom):
        self.nom = nom

    def getNom(self):
        return self.nom

    def setPrenom(self, prenom):
        self.prenom = prenom

    def getPrenom(self):
        return self.prenom
```

Exemple_v2 : classe Personne

```
25
26      def setAge(self, age):
27          self.age = age
28
29      def getAge(self):
30          return self.age
31
32      # méthode
33      def affiche(self):
34          print("Nom : ", self.nom, ", Prenom : ", self.prenom, ", age : ", self.age)
35
```

Exemple v2 : classe Etudiant

etudiant.py

```python
from personne import Personne

class Etudiant(Personne):
    numEtd = None

    # Constructeur de la classe
    def __init__(self, nom, prenom, age, numEtd):
        super().__init__(nom, prenom, age)
        self.numEtd = numEtd

    # Accesseur de l'attribut age
    def setNumEtd(self, numEtd):
        self.numEtd = numEtd

    def getNumEtd(self):
        return self.numEtd

    # Rédefinition de méthode
    def affiche(self):
        super().affiche()
        print("NumEtd : ", self.numEtd)
```

Exemple v2 : classe Enseignant

enseignant.py

```
1   from personne import Personne
2
3   class Enseignant(Personne):
4       specialite = None
5       grade = None
6       salaire = None
7
8       # Constructeur de la classe
9       def __init__(self, nom, prenom, age, specialite, grade, salaire):
10          super().__init__(nom, prenom, age)
11          self.specialite = specialite
12          self.grade = grade
13          self.salaire = salaire
14
15      # Accesseurs
16      def setSpecialite(self, specialite):
17          self.specialite = specialite
18
19      def getSpecialite(self):
20          return self.specialite
21
22      def setGrade(self, grade):
23          self.grade = grade
```

Exemple_v2 : classe Enseignant

```
23          self.grade = grade
24
25      def getGrade(self):
26          return self.grade
27
28      def setSalaire(self, salaire):
29          self.salaire = salaire
30
31      def getSalaire(self):
32          return self.salaire
33
34      # Rédefinition de méthode
35      def affiche(self):
36          super().affiche()
37          print(", specialité : ", self.specialite,", grade : ", self.grade,", salaire : ", self.salaire)
```

Exemple_v2 : main.py

main.py

```
1  from etudiant import Etudiant
2  from enseignant import Enseignant
3
4  # Programme Principal
5  Et1 = Etudiant("ALOUI", "kais", 30, 587)
6  En1 = Enseignant("KHEMIRI", "Amira", 29, "Informatique", "MA", 2500)
7  En1.setSalaire(2600)
8  Et1.affiche()
9  En1.affiche()
```

```
Nom :  ALOUI , Prenom :  kais , age :  30
NumEtd :  587
Nom :  KHEMIRI , Prenom :  Amira , age :  29
, specialité :  Informatique , grade :  MA , salaire :  2600
```

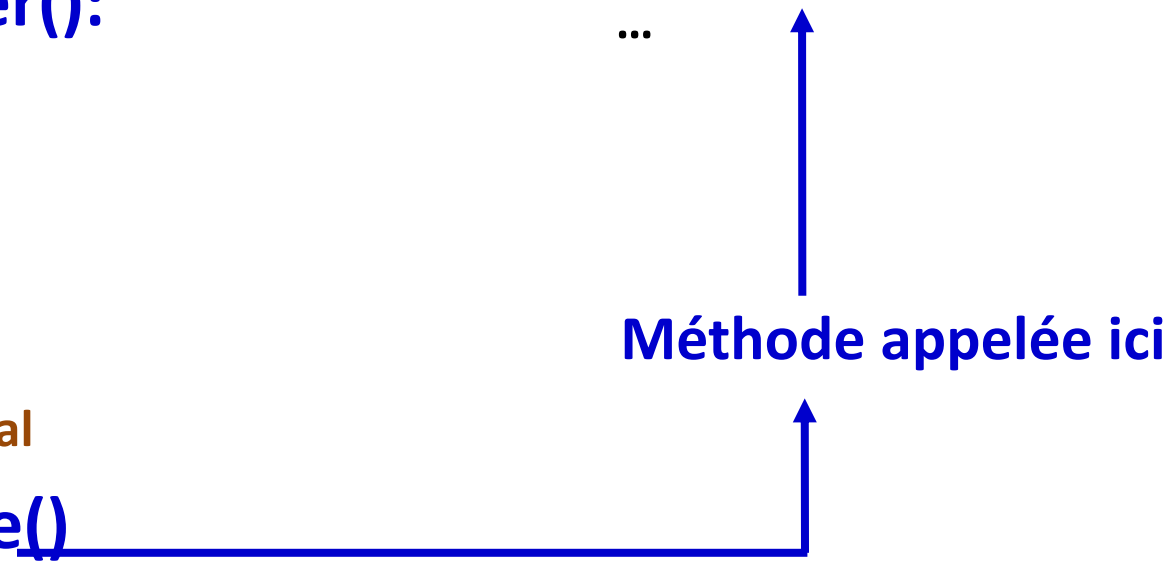
Appel de méthodes

- **Exemple 1/2:**

```
class Vehicule:
    def demarrer():
    ...

class Voiture (Vehicule):
    def demarrer():
    ...

# Programme principal
    v = Voiture()
    v.demarrer()
}
}
```



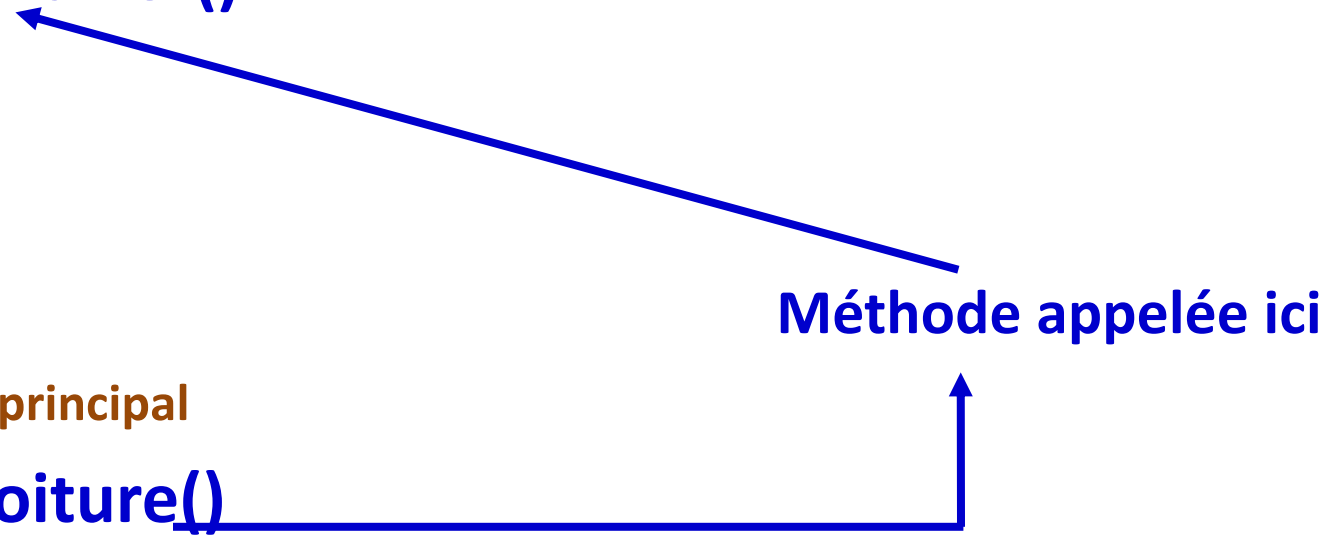
Appel de méthodes

- **Exemple 1/2:**

```
class Vehicule:
    def demarrer():
    ...

class Voiture (Vehicule):
    ...

# Programme principal
    v = Voiture()
    v.demarrer()
}
}
```

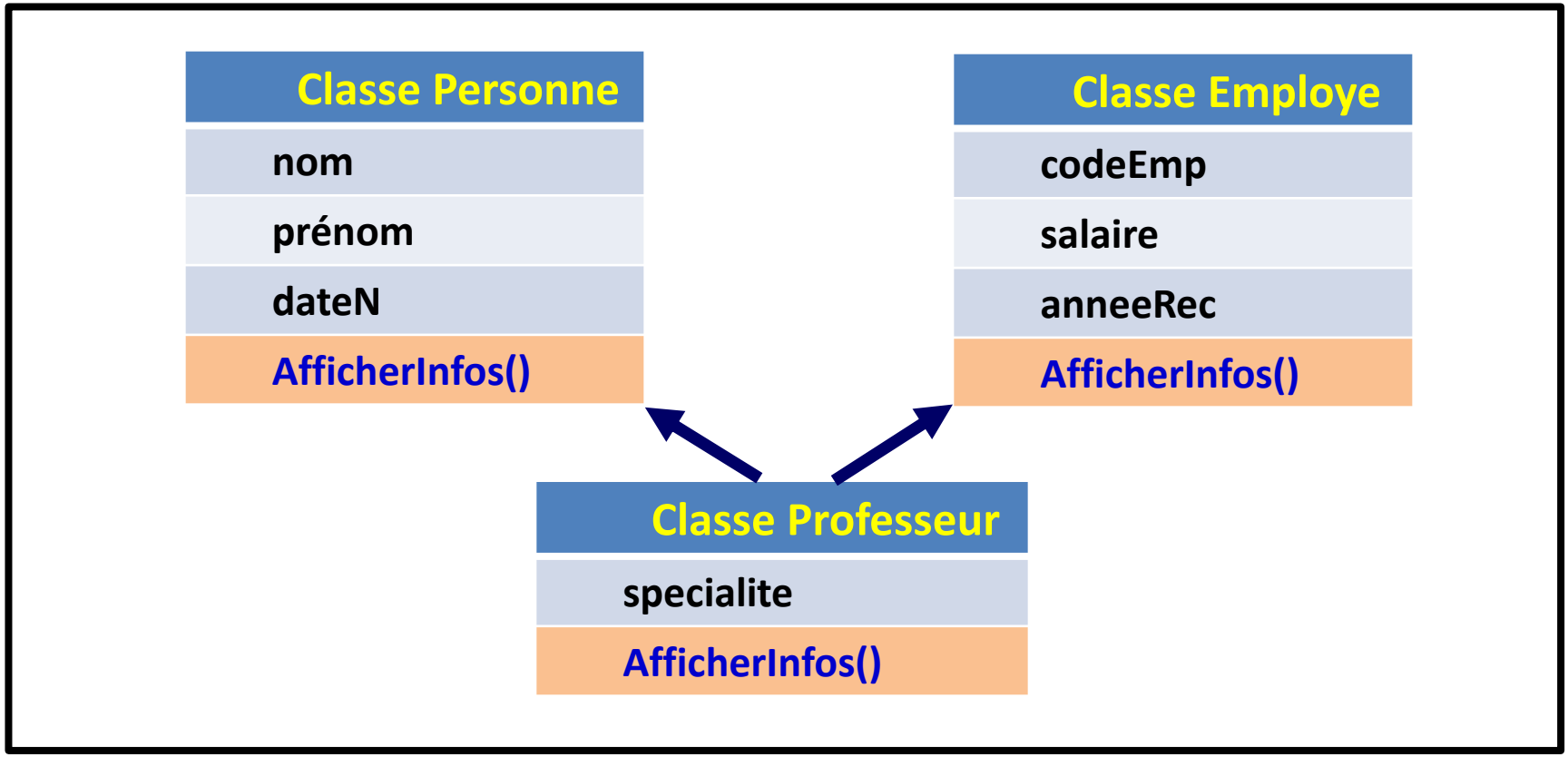


- **Si la méthode n'existe pas dans la classe de l'objet créé, Le programme va chercher dans la classe mère.**

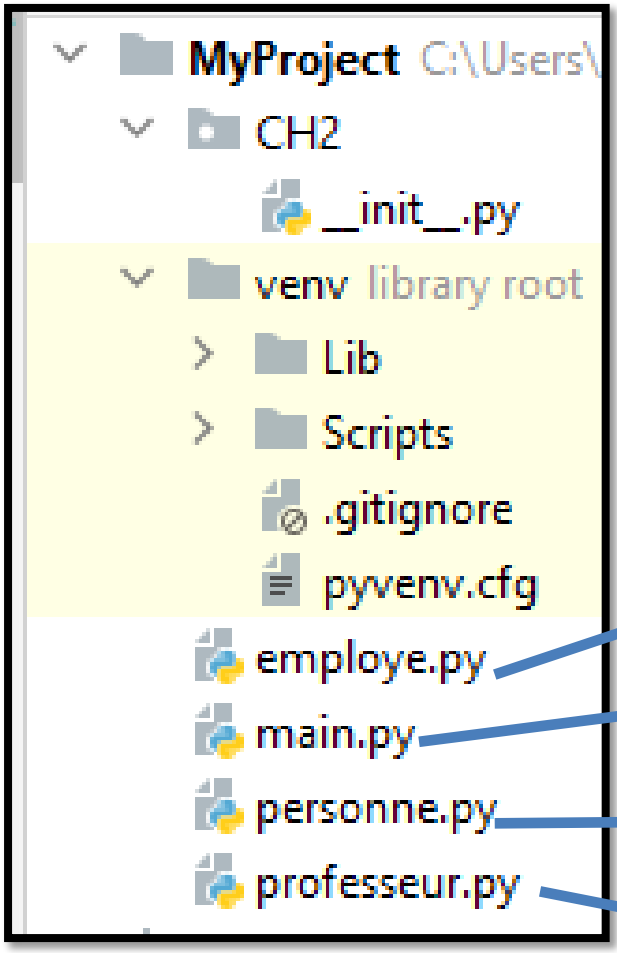
Héritage multiple

- **Python autorise l'héritage multiple.**

- **L'héritage multiple est le processus ou une classe fille hérite des attributs et des méthodes de plusieurs classes mères .**



Héritage multiple_v1



Classe Employe

Programme principal

Classe Personne

Classe Professeur

Héritage multiple_v1 : classe Personne

personne.py

```python
class Personne:
    # Attributs
    nom = None
    prenom = None
    dateN = None

    # Constructeur de la classe
    def __init__(self, nom, prenom, dateN):
        self.nom = nom
        self.prenom = prenom
        self.dateN = dateN

    # Accesseurs
    def setNom(self, nom):
        self.nom = nom

    def getNom(self):
        return self.nom

    def setPrenom(self, prenom):
        self.prenom = prenom

    def getPrenom(self):
        return self.prenom
```

Héritage multiple_v1 : classe Personne

```
25
26      def setDateN(self, dateN):
27          self.dateN = dateN
28
29      def getDateN(self):
30          return self.dateN
31
32      # méthode
33      def afficherInfos(self):
34          print("Nom : ", self.nom, ", Prenom : ", self.prenom, ", Date naissance : ", self.dateN)
35
```

Héritage multiple_v1 : classe Employe

employe.py

```
1  class Employe:
2      # Attributs
3      codeEmp = None
4      salaire = None
5      anneeRec = None
6
7      # Constructeur de la classe
8      def __init__(self, codeEmp, salaire, anneeRec):
9          self.codeEmp = codeEmp
10         self.salaire = salaire
11         self.anneeRec = anneeRec
12
13     # Accesseurs
14     def setCodeEmp(self, codeEmp):
15         self.codeEmp = codeEmp
16
17     def getCodeEmp(self):
18         return self.codeEmp
19
20     def setSalaire(self, salaire):
21         self.salaire = salaire
22
```

Héritage multiple_v1 : classe Employe

```
23      def getSalaire(self):
24          return self.salaire
25
26      def setAnneeRec(self, anneeRec):
27          self.anneeRec = anneeRec
28
29      def getAnneeRec(self):
30          return self.anneeRec
31
32      # méthode
33      def afficherInfos(self):
34          print("Code employé : ", self.codeEmp, ", Salaire : ", self.salaire, ", anneeRec : ", self.anneeRec)
35
```

Héritage multiple_v1 : classe professeur

professeur.py

```
1   from personne import Personne
2   from employe import Employe
3
4   class Professeur(Personne, Employe):
5       # Attributs
6       specialite = None
7
8       # Constructeur de la classe
9       def __init__(self, nom, prenom, dateN, codeEmp, salaire, anneeRec, specialite):
10          Personne.__init__(self, nom, prenom, dateN)
11          Employe.__init__(self, codeEmp, salaire, anneeRec)
12          self.specialite = specialite
13
14      # Accesseurs
15      def setSpecialite(self, specialite):
16          self.specialite = specialite
17
18      def getSpecialite(self):
19          return self.specialite
20
```

Héritage multiple_v1 : classe professeur

```
21      # méthode
22      def afficherInfos(self):
23          Personne.afficherInfos(self)
24          Employe.afficherInfos(self)
25          print(", Specialite : ", self.specialite)
```

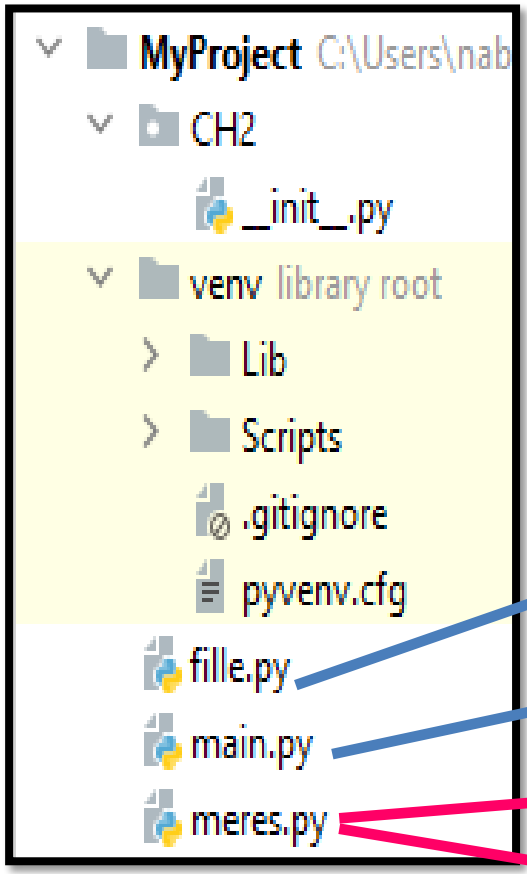
Héritage multiple_v1 : main.py

main.py

```
1  from professeur import Professeur
2
3  P1 = Professeur("SOULI", "Slim", "09/10/1985", 123, 1600, 2020,"informatique")
4  P1.afficherInfos()
5
```

```
Nom :  SOULI , Prenom :  Slim , Date naissance :  09/10/1985
Code employé :  123 , Salaire :  1600 , anneeRec :  2020
, Specialite :  informatique
```

Héritage multiple_v2



Classe Professeur

Programme principal

Classe Personne

Classe Employe

Héritage multiple_v2 : meres.py

meres.py

```python
class Personne:
    # Attributs
    nom = None
    prenom = None
    dateN = None

    # Constructeur de la classe
    def __init__(self, nom, prenom, dateN):
        self.nom = nom
        self.prenom = prenom
        self.dateN = dateN

    # Accesseurs
    def setNom(self, nom):
        self.nom = nom

    def getNom(self):
        return self.nom

    def setPrenom(self, prenom):
        self.prenom = prenom

    def getPrenom(self):
        return self.prenom
```

Héritage multiple_v2 : meres.py

```
26      def setDateN(self, dateN):
27          self.dateN = dateN
28
29      def getDateN(self):
30          return self.dateN
31
32      # méthode
33      def afficherInfos(self):
34          print("Nom : ", self.nom, ", Prenom : ", self.prenom, ", Date naissance : ", self.dateN)
35  class Employe:
36      # Attributs
37      codeEmp = None
38      salaire = None
39      anneeRec = None
40
41      # Constructeur de la classe
42      def __init__(self, codeEmp, salaire, anneeRec):
43          self.codeEmp = codeEmp
44          self.salaire = salaire
45          self.anneeRec = anneeRec
46
47      # Accesseurs
48      def setCodeEmp(self, codeEmp):
49          self.codeEmp = codeEmp
```

Héritage multiple_v2 : meres.py

```
50
51      def getCodeEmp(self):
52          return self.codeEmp
53
54      def setSalaire(self, salaire):
55          self.salaire = salaire
56
57      def getSalaire(self):
58          return self.salaire
59
60      def setAnneeRec(self, anneeRec):
61          self.anneeRec = anneeRec
62
63      def getAnneeRec(self):
64          return self.anneeRec
65
66      # methode
67      def afficherInfos(self):
68          print("Code employé : ", self.codeEmp, ", Salaire : ", self.salaire, ", anneeRec : ", self.anneeRec)
```

Héritage multiple_v2 : fille.py

fille.py

```python
from meres import Personne
from meres import Employe

class Professeur(Personne, Employe):
    # Attributs
    specialite = None

    # Constructeur de la classe
    def __init__(self, nom, prenom, dateN, codeEmp, salaire, anneeRec, specialite):
        Personne.__init__(self, nom, prenom, dateN)
        Employe.__init__(self, codeEmp, salaire, anneeRec)
        self.specialite = specialite

    # Accesseurs
    def setSpecialite(self, specialite):
        self.specialite = specialite

    def getSpecialite(self):
        return self.specialite

```

Héritage multiple_v2 : fille.py

```
20
21      # méthode
22      def afficherInfos(self):
23          Personne.afficherInfos(self)
24          Employe.afficherInfos(self)
25          print(", Specialite : ", self.specialite)
26
```

Héritage multiple_v2 : main.py

main.py

```
1  from professeur import Professeur
2
3  P1 = Professeur("SOULI", "Slim", "09/10/1985", 123, 1600, 2020,"informatique")
4  P1.afficherInfos()
5
```

```
Nom :  SOULI , Prenom :  Slim , Date naissance :  09/10/1985
Code employé :  123 , Salaire :  1600 , anneeRec :  2020
, Specialite :  informatique
```

d. Les classes abstraites

Classes et méthodes abstraites

- **Une classe abstraite** contient **au moins une méthode abstraite.**

- **Méthode abstraite:** on **déclare que son en-tête** (**n'admettant pas d'implémentation**).

Syntaxe :

```
def nomMéthode(self):
    pass
```

- **Les classes abstraites** peuvent également **contenir des méthodes déjà implémentés (concrètes)**.

- **Une classe abstraite est non instanciable directement. Elle doit être dérivée.**

Classes et méthodes abstraites

- **Une classe abstraite doit avoir des classes filles concrètes.**

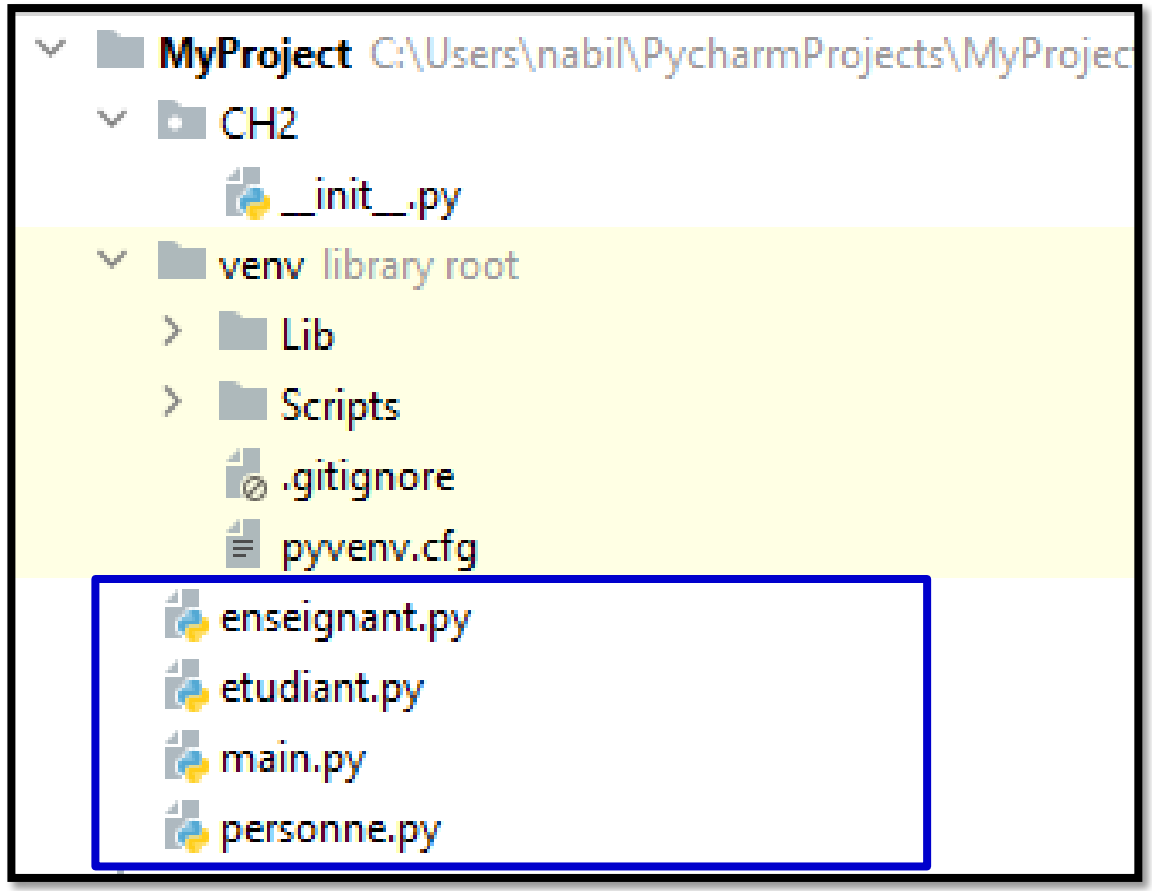
- **Les classes abstraites** permettent de **créer des classes génériques expliquant certains comportements sans les implémenter.**

- Une classe abstraite **propose un certain nombre** de **fonctionnalités similaires pour tous ses futurs descendants.**

- Les classes abstraites sont **un outil intéressant** pour le **polymorphisme (la redéfinition de méthodes dans les classes filles).**

- Une classe fille d'une classe abstraite **n'est pas obligée d'implémenter toutes les méthodes abstraites** de sa classe mère.

Exemple :


MyProject C:\Users\nabil\PycharmProjects\MyProjec
CH2
__init__.py
venv library root
Lib
Scripts
.gitignore
pyvenv.cfg
enseignant.py
etudiant.py
main.py
personne.py

Exemple : classe Personne

personne.py

```python
# abc est un module python intégré, nous importons ABC et abstractmethod
from abc import ABC, abstractmethod

class Personne(ABC): # hériter de ABC(Abstract Base Class)
    nom = None
    prenom = None
    age = None

    # Constructeur de la classe
    def __init__(self, nom, prenom, age):
        self.nom = nom
        self.prenom = prenom
        self.age = age

    # Accesseur de l'attribut age
    def setAge(self, age):
        self.age = age
    def getAge(self):
        return self.age

    @abstractmethod    # un décorateur pour définir une méthode abstraite
    def affiche(self):
        pass
```

Exemple : classe Etudiant

etudiant.py

```python
from personne import Personne

class Etudiant(Personne):
    numEtd = None

    # Constructeur de la classe
    def __init__(self, nom, prenom, age, numEtd):
        super().__init__(nom, prenom, age)
        self.numEtd = numEtd

    # Accesseurs
    def setNumEtd(self, numEtd):
        self.numEtd = numEtd

    def getNumEtd(self):
        return self.numEtd

    # méthode
    def affiche(self):
        print("Nom : ", self.nom, ", Prenom : ", self.prenom, ", age : ", self.age)
        print(",NumEtd : ", self.numEtd)
```

Exemple : classe Enseignant

enseignant.py

```python
from personne import Personne

class Enseignant(Personne):
    specialite = None
    grade = None
    salaire = None

    # Constructeur de la classe
    def __init__(self, nom, prenom, age, specialite, grade, salaire):
        super().__init__(nom, prenom, age)
        self.specialite = specialite
        self.grade = grade
        self.salaire = salaire

    # Accesseurs
    def setSpecialite(self, specialite):
        self.specialite = specialite

    def getSpecialite(self):
        return self.specialite

    def setGrade(self, grade):
        self.grade = grade
```

Exemple : classe Enseignant

```
24
25      def getGrade(self):
26          return self.grade
27
28      def setSalaire(self, salaire):
29          self.salaire = salaire
30
31      def getSalaire(self):
32          return self.salaire
33      # méthode
34      def affiche(self):
35          print("Nom : ", self.nom, ", Prenom : ", self.prenom, ", age : ", self.age)
36          print(", specialité : ", self.specialite,", grade : ", self.grade,", specialité : ", self.salaire)
37
```

Exemple_v1 : main.py

main.py

```python
from personne import Personne
from etudiant import Etudiant
from enseignant import Enseignant

# Programme Principal
P1 = Personne("SALHI", "Farah", 22)
Et1 = Etudiant("ALOUI", "kais", 30, 587)
En1 = Enseignant("KHEMIRI", "Amira", 29, "Informatique", "MA", 2500)
En1.setSalaire(2600)

Et1.affiche()
En1.affiche()
```

```
Traceback (most recent call last):
  File "C:\Users\nabil\PycharmProjects\MyProject\main.py", line 6, in <module>
    P1 = Personne("SALHI", "Farah", 22)
         ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
TypeError: Can't instantiate abstract class Personne with abstract method affiche
```

Exemple_v2 : main.py

main.py

```
1  from etudiant import Etudiant
2  from enseignant import Enseignant
3
4  # Programme Principal
5  Et1 = Etudiant("ALOUI", "kais", 30, 587)
6  En1 = Enseignant("KHEMIRI", "Amira", 29, "Informatique", "MA", 2500)
7  En1.setSalaire(2600)
8
9  Et1.affiche()
10 En1.affiche()
```

```
Nom :  ALOUI , Prenom :  kais , age :  30
,NumEtd :  587
Nom :  KHEMIRI , Prenom :  Amira , age :  29
, specialité :  Informatique , grade :  MA , specialité :  2600
```

e. Les interfaces

Interface

- Si l'on considère **une classe abstraite** qui ne content **aucune méthode concrète** on aboutit à la notion d'**interface**.

- **Une interface contient** que **les déclarations des méthodes abstraites.**

- **Une classe qui implémente une interface** : elle fournit **obligatoirement les implantations** de **toutes les méthodes abstraites déclarées dans l'interface** .

Interface

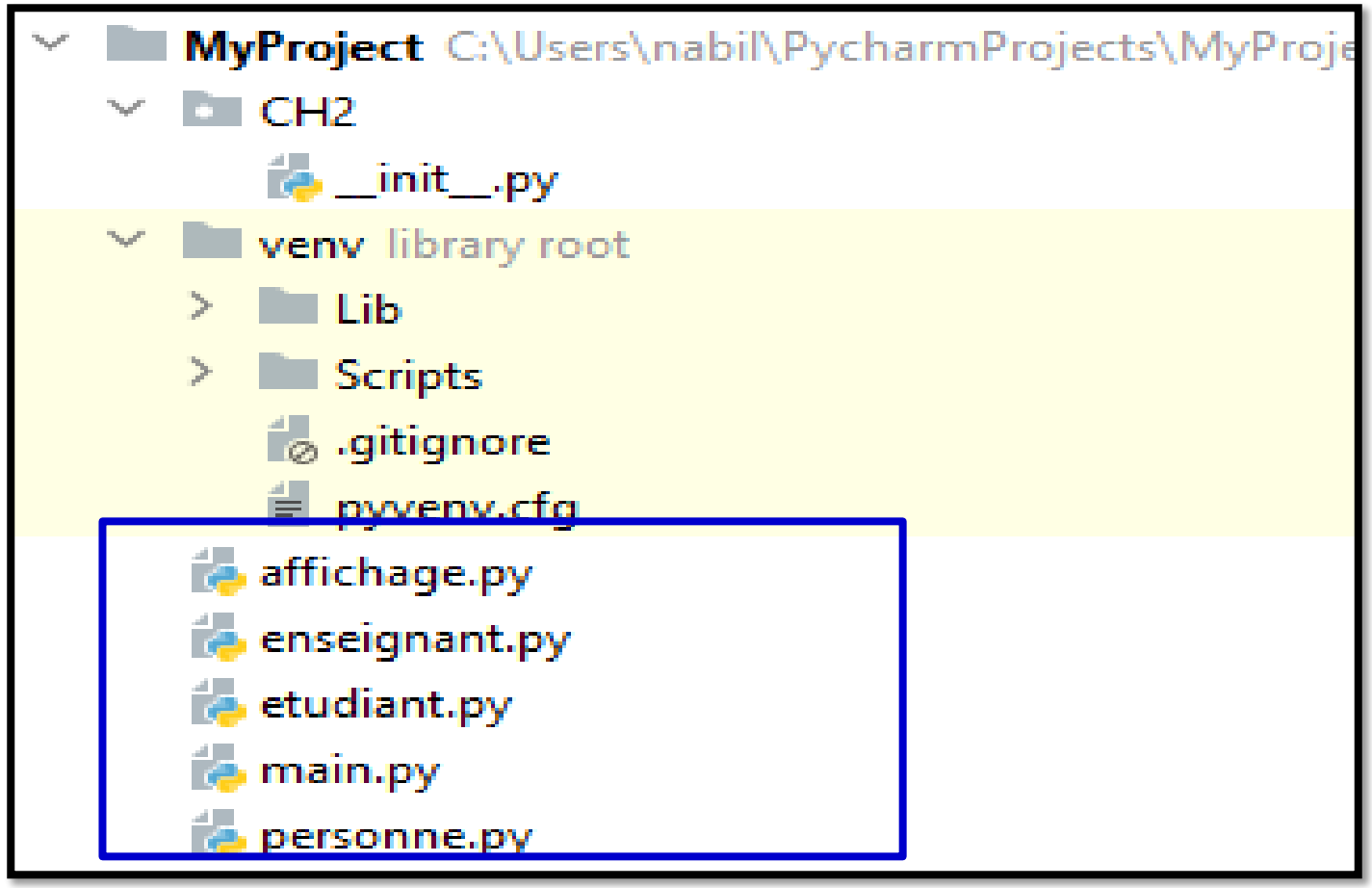
- Utilisée **à la place** d'une **classe abstraite** quand il n'y a **aucune méthode concrète et aucun attribut.**

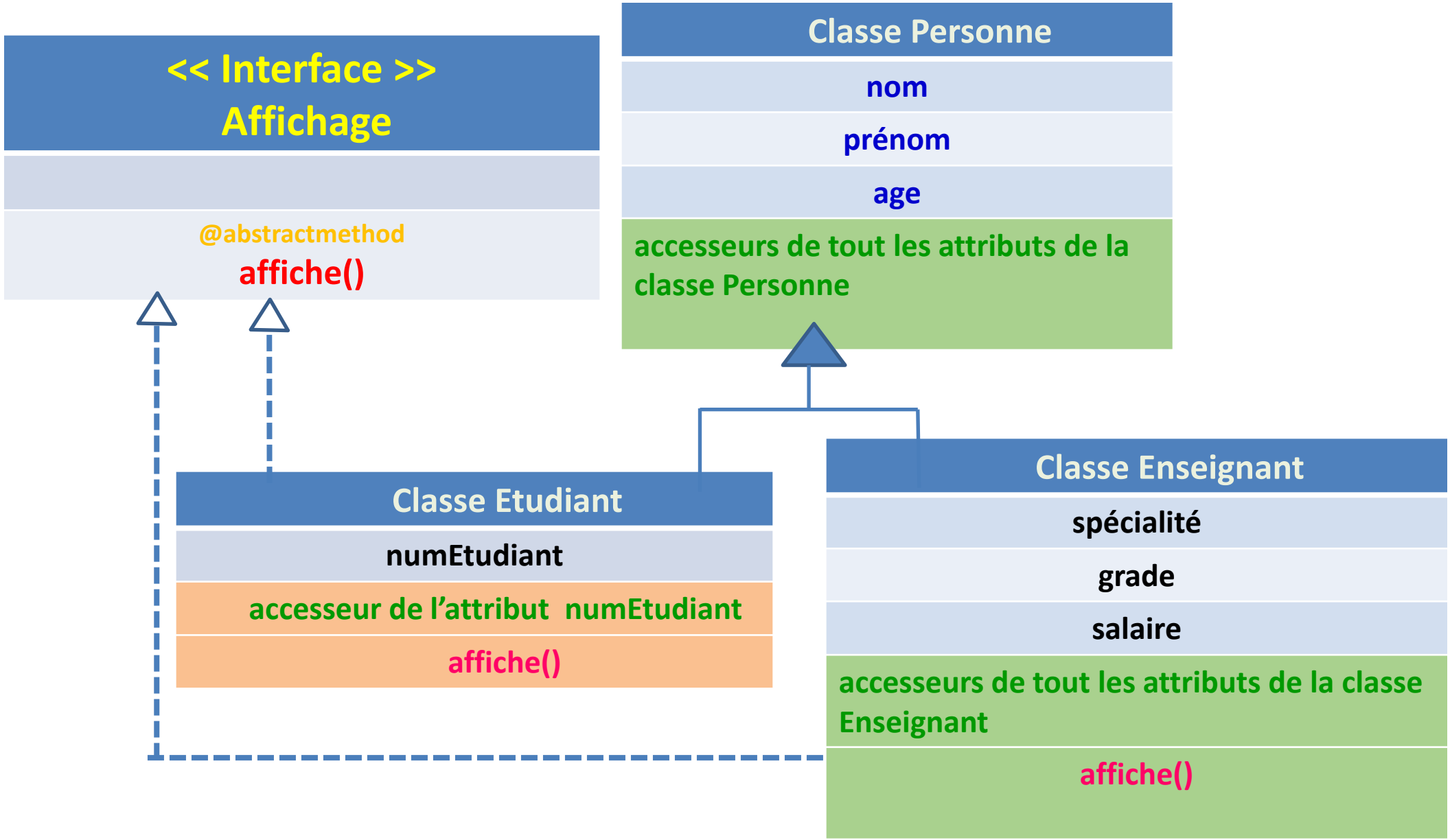
- **Une interface ne peut pas avoir de constructeur(s). Une interface est non instanciable. La construction d'un objet nécessite une classe qui implémente l'interface.**

- **Une classe qui implémente une interface peut avoir ses propres attributs** et **méthodes**.

- **Une classe peut implémenter plusieurs interfaces**

Exemple 1


MyProject C:\Users\nabil\PycharmProjects\MyProje
CH2
__init__.py
venv library root
Lib
Scripts
.gitignore
pyvenv.cfg
affichage.py
enseignant.py
etudiant.py
main.py
personne.py


<< Interface >>
Affichage
@abstractmethod
affiche()
Classe Personne
nom
prénom
age
accesseurs de tout les attributs de la classe Personne
Classe Etudiant
numEtudiant
accesseur de l'attribut numEtudiant
affiche()
Classe Enseignant
spécialité
grade
salaire
accesseurs de tout les attributs de la classe Enseignant
affiche()

Exemple : InterfaceAffichage

affichage.py

```python
# abc est un module python intégré, nous importons ABC et abstractmethod
from abc import ABC, abstractmethod


class InterfaceAffichage(ABC):  # hériter de ABC(Abstract Base Class)

    @abstractmethod
    def affiche(self):
        pass
```

Exemple : Classe Personne

personne.py

```python
class Personne:
    nom = None
    prenom = None
    age = None

    # Constructeur de la classe
    def __init__(self, nom, prenom, age):
        self.nom = nom
        self.prenom = prenom
        self.age = age

    # Accesseur de l'attribut age
    def setAge(self, age):
        self.age = age

    def getAge(self):
        return self.age
```

Exemple : Classe Etudiant

etudiant.py

```python
from personne import Personne
from affichage import InterfaceAffichage

class Etudiant(Personne, InterfaceAffichage):
    numEtd = None

    # Constructeur de la classe
    def __init__(self, nom, prenom, age, numEtd):
        super().__init__(nom, prenom, age)
        self.numEtd = numEtd

    # Accesseurs
    def setNumEtd(self, numEtd):
        self.numEtd = numEtd

    def getNumEtd(self):
        return self.numEtd

    # méthode
    def affiche(self):
        print("Nom : ", self.nom, ", Prenom : ", self.prenom, ", age : ", self.age)
        print(",NumEtd : ", self.numEtd)
```

Exemple : Classe Enseignant

enseignant.py

```python
from personne import Personne
from affichage import InterfaceAffichage

class Enseignant(Personne, InterfaceAffichage):
    specialite = None
    grade = None
    salaire = None

    # Constructeur de la classe
    def __init__(self, nom, prenom, age, specialite, grade, salaire):
        super().__init__(nom, prenom, age)
        self.specialite = specialite
        self.grade = grade
        self.salaire = salaire

    # Accesseurs
    def setSpecialite(self, specialite):
        self.specialite = specialite

    def getSpecialite(self):
        return self.specialite
```

Exemple : Classe Enseignant

```
22
23      def setGrade(self, grade):
24          self.grade = grade
25
26      def getGrade(self):
27          return self.grade
28
29      def setSalaire(self, salaire):
30          self.salaire = salaire
31
32      def getSalaire(self):
33          return self.salaire
34      # méthode
35      def affiche(self):
36          print("Nom : ", self.nom, ", Prenom : ", self.prenom, ", age : ", self.age)
37          print(", specialité : ", self.specialite,", grade : ", self.grade,", specialité : ", self.salaire)
38
```

Exemple : main.py

main.py

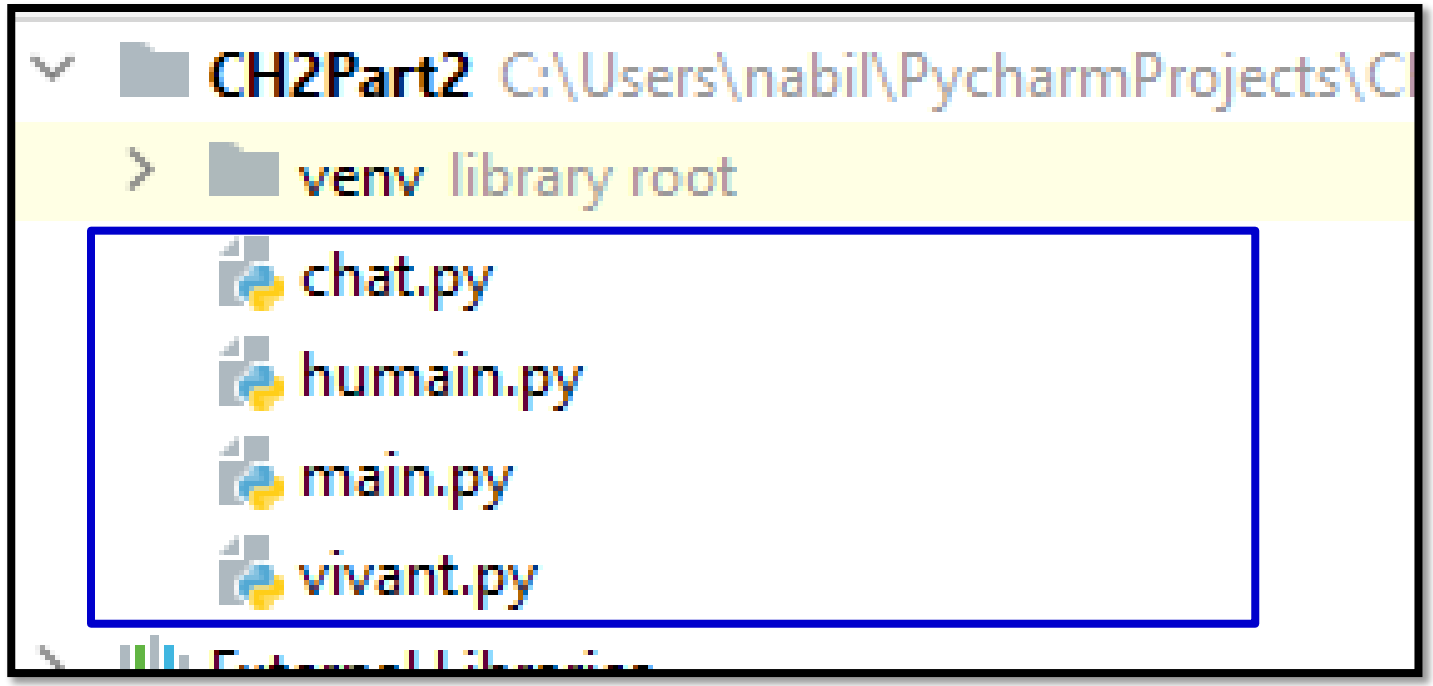
```python
from etudiant import Etudiant
from enseignant import Enseignant

# Programme Principal
Et1 = Etudiant("ALOUI", "kais", 30, 587)
En1 = Enseignant("KHEMIRI", "Amira", 29, "Informatique", "MA", 2500)
En1.setSalaire(2600)

Et1.affiche()
En1.affiche()
```

```
Nom :  ALOUI , Prenom :  kais , age :  30
,NumEtd :  587
Nom :  KHEMIRI , Prenom :  Amira , age :  29
, specialité :  Informatique , grade :  MA , specialité :  2600
```

Exemple 2


CH2Part2 C:\Users\nabil\PycharmProjects\CI
venv library root
chat.py
humain.py
main.py
vivant.py

Exemple 2

vivant.py

```
1   from abc import ABC, abstractmethod
2
3   class Vivant(ABC):
4
5       @abstractmethod
6       def enVie(self):
7           pass
8
9       @abstractmethod
10      def mourir(self):
11          pass
12
13      @abstractmethod
14      def reanimer(self):
15          pass
```

Exemple 2

humain.py

```python
from vivant import Vivant

class Humain(Vivant):
    etat = None

    def __init__(self):
        self.etat = True

    def enVie(self):
        return self.etat

    def mourir(self):
        if self.etat is True:
            self.etat = False
            print("L'être humain est mort!")
        else:
            print("L'être humain est déjà mort!")

    def reanimer(self):
        if self.etat is True:
            print("Cet Humain est toujours en vie.")
        else:
            print("Impossible de réanimer cet Humain il est déjà mort!")
```

Exemple 2

chat.py

```python
from vivant import Vivant

class Chat(Vivant):
    etat = None
    nbVies = None

    def __init__(self):
        self.etat = True
        self.nbVies = 7

    def enVie(self):
        return self.etat

    def mourir(self):
        if self.etat is True:
            self.etat = False
            print("Le chat est en train de mourir!")
        else:
            print("Le chat est mort!")
```

Exemple 2

```
20
21      def reanimer(self):
22          if self.etat is True:
23              print("Ce chat est toujours en vie.")
24          elif self.etat is False and self.nbVies > 0:
25              self.etat = True
26              self.nbVies = self.nbVies - 1
27          else:
28              print("Impossible de réanimer cet chat il est déjà mort!")
29
```

Exemple 2

main.py

```
1  from humain import Humain
2  from chat import Chat
3
4  H1 = Humain()
5  C1 = Chat()
6
7  print("Est-il vivant? : ", H1.enVie())
8  H1.mourir()
9  H1.reanimer()
10 H1.reanimer()
11 H1.mourir()
12 H1.reanimer()
13
14 print("**********************************\n")
15
16 print("Est-il vivant? : ", C1.enVie())
17 C1.mourir()
18 C1.reanimer()
19 C1.reanimer()
20 C1.mourir()
21 C1.reanimer()
```

Exemple 2 : Exécution

```
Est-il vivant? :  True
L'être humain est mort!
Impossible de réanimer cet Humain il est déjà mort!
Impossible de réanimer cet Humain il est déjà mort!
L'être humain est déjà mort!
Impossible de réanimer cet Humain il est déjà mort!
**********************************

Est-il vivant? :  True
Le chat est en train de mourir!
Ce chat est toujours en vie.
Le chat est en train de mourir!
```

1.3 La gestion des exceptions en Python

A suivre …